

集群操作指南 V0.2

编写：Golden Pigeon 校对：lmszs
以及实验室的大家

October 08, 2025

本指南的编写目的是帮助实验室的成员安全，高效地利用实验室集群。本手册仍有不完善之处，如有需要补充之处，请联系本手册作者。微信：GP13397203339 😊

本手册为内部资料，请勿外传，如有泄露，后果自负。🙊

本手册总计 17525 字

目录

1. 集群结构	5
1.1. 整体结构	5
1.2. 存储结构	5
1.2.1. 使用技巧与建议	6
2. 用户	8
2.1. 用户登录	8
2.1.1. ssh 命令密码登录	8
2.1.2. ssh 命令秘钥登录	8
2.1.3. 配置 ssh 一键登录	9
2.1.4. 使用 VSCode Remote-SSH 登录	9
2.1.5. 高级：在校园网外登录集群	10
2.2. 用户创建（管理员必看）	11
3. 网络	13
3.1. 集群访问互联网	13
3.2. 访问集群网络服务	13
4. 实验	15
4.1. 实验环境	15
4.1.1. Python	15
4.1.2. CUDA	15
4.1.3. C/C++	20
4.1.3.1. 概念辨析	20
4.1.3.2. 编译	21
4.1.3.3. 构建	22
4.1.3.3.1. make	22
4.1.3.3.2. autotools	24
4.1.3.3.3. CMake	26
4.2. 实验后台执行	28
4.2.1. 基于 SLURM 集群任务调度工具的实验（待补充）	29
4.2.2. 基于终端复用器的实验	29
5. Git	31
5.1. 概念	31
5.2. 项目结构	32
5.3. 版本控制实战	33
5.4. 一个自建的 GitLab 服务器	52
Appendix	53
A. Linux 使用技巧	53
A.1. 文件操作	53
A.2. 压缩与解压	54
B. 一个自建的基于 Wireguard 的 VPN 网络	57
B.1. 加入网络	57
B.1.1. Windows	57
B.1.2. Linux	58

B.1.3. MacOS	59
B.2. VPN 网络注意事项	59
Index of Figures	60
Index of Tables	60
Index of Listings	60

1. 集群结构

1.1. 整体结构

VRL 实验室的集群由 1 个主节点和 11 个计算节点构成。其中，主节点不含 GPU，而计算节点中装有不同数量不同型号的 GPU。各节点的中文名称，英文名称，外网 IP 地址，内网 IP 地址及 GPU 安装如表 1 所示。

节点中文名称	节点名称	管理 IP	内网 IP	显卡
主节点	mgr	10.171.92.44	10.20.30.10	N/A
节点 1	node01	10.171.92.1	10.20.30.11	2080Ti × 10
节点 2	node02	10.171.92.2	10.20.30.12	2080Ti × 10
节点 3	node03	10.171.92.3	10.20.30.13	2080Ti × 10
节点 4	node04	10.171.92.4	10.20.30.14	2080Ti × 10
节点 5	node05	10.171.92.5	10.20.30.15	2080Ti × 10
节点 6	node06	10.171.92.49	10.20.30.16	3080 × 5 + 3090 × 5
节点 7	node07	10.171.92.6	10.20.30.17	3090 × 8
节点 8	node08	10.171.92.46	10.20.30.18	3090 × 10
节点 9	node09	10.171.92.51	10.20.30.19	3090 × 10
节点 10	node10	10.171.92.20	10.20.30.20	4090 × 8
节点 11	node11	10.171.92.23	10.20.30.21	4090 × 8

表 1 集群整体结构

1.2. 存储结构

集群中的每一个节点（包含主节点）中都有一块 SSD 系统盘，其中主节点的系统盘为 4T，其他节点的系统盘为 1T，均挂载在系统根目录 / 下。除此以外，集群还拥有数据存储，分为共享存储和节点本地存储，其中，共享存储由挂载在主节点上的 RAID0 磁盘阵列和一块固态硬盘提供，节点本地存储由各节点本地磁盘提供。主节点的磁盘阵列由 6 块 16T 的机械硬盘组成，阵列的总容量为 87T，挂载于 /data 目录下，另有一 8T 固态硬盘，挂载于 /SSD 目录下，上述共享存储通过 NFS 协议共享给其他节点，挂载于各节点的 /data 和 /SSD 目录下。各节点拥有一块或两块本地机械硬盘，容量均为 4T，挂载于 /HDD1 或 /HDD2 目录下。各种存储方式，其挂载目录及其特点如表 2 所示。

存储属性	系统盘	共享存储	节点本地存储
存储形式	SSD	RAID0 HDD 阵列，NFS	HDD
挂载目录	/	/data 和/SSD	/HDD1 或/HDD2
容量	主节点 4T，其他节点 1T	6 × 16T HDD RAID0 87T, 8T SSD	4T/块

速度	快	慢	中等
优点	读写速度快，系统盘	容量大，所有节点可访问	均衡表现
缺点	容量小，容易占满	速度慢，延迟高，且同时访问用户较多时会进一步变慢，若存放需要高频访问的大容量数据集，可能严重拖慢实验速度	均衡表现
使用建议	存放软件，运行环境，代码以及需要高频读取但空间占用较低的数据	存放未处理的或无需高频访问的数据集，模型权重，实验日志，实验产生的中间文件等	存放一般的数据集

表 2 集群存储结构

1.2.1. 使用技巧与建议

- `df -h`：查看各挂载点的磁盘使用情况

示例：

\$ df -h

sh

```
Filesystem      Size  Used Avail Use% Mounted on
udev            126G   0    126G   0% /dev
tmpfs           26G   4.6M   26G    1% /run
/dev/sda2       877G  757G   75G   91% /
tmpfs           126G   4.5M  126G    1% /dev/shm
tmpfs           5.0M   4.0K   5.0M    1% /run/lock
tmpfs           126G   0    126G    0% /sys/fs/cgroup
/dev/loop0      128K  128K     0 100% /snap/bare/5
/dev/loop1     347M  347M     0 100% /snap/gnome-3-38-2004/119
/dev/loop2       92M   92M     0 100% /snap/gtk-common-themes/1535
/dev/loop3       64M   64M     0 100% /snap/core20/1828
/dev/loop4     350M  350M     0 100% /snap/gnome-3-38-2004/143
/dev/loop5       50M   50M     0 100% /snap/snapd/18357
/dev/loop6       46M   46M     0 100% /snap/snap-store/638
/dev/loop7       45M   45M     0 100% /snap/snapd/23545
/dev/sdc        3.6T  3.4T   62G   99% /HDD2
/dev/sdb        3.6T  1.4T  2.1T   40% /HDD1
/dev/sda1       2.8G   6.1M   2.8G    1% /boot/efi
tmpfs           26G   24K   26G    1% /run/user/126
tmpfs           26G   12K   26G    1% /run/user/1007
mgr:/SSD        7.0T  5.8T  826G   88% /SSD
tmpfs           26G   64K   26G    1% /run/user/1000
tmpfs           26G   4.0K   26G    1% /run/user/1002
```

tmpfs	26G	4.0K	26G	1%	/run/user/1041
mgr:/data	87T	17T	67T	20%	/data

其中需要注意的是节点本地系统盘 `/dev/sda2`，节点本地存储 `/dev/sdb` 和 `/dev/sdc`，以及共享存储 `mgr:/data` 和 `mgr:/SSD` 的使用情况。在某块盘快占满时，应考虑将多余的数据清除或转移到其他盘。

- `du -sh /data/your_folder`：查看某个目录所占空间，并按人类可读形式（B, K, M, G, T）显示

通常用法：

```
# 查看所有用户目录所占空间大小并且忽视报错（无法访问等），并从大到小排序，
du -sh /home/* 2> /dev/null | sort -rh
```

```
255G /home/zhuyufan
109G /home/zhengjingzhu
34G /home/liangzhechun
21G /home/ranchingzhi
1.2G /home/hpc
```

当发现盘快占满且某个用户占用较多空间时，请截图该命令执行结果，并联系该用户清理其目录下不必要的数据。

- `ln -s /absolute/path/to/your/folder path/to/your/folder`：创建符号链接

有时你希望能直接在项目代码目录中访问实验日志、数据集等，但又不希望将这些大文件直接存放在项目目录中（这样会占用较多系统盘空间），你可以使用符号链接将共享存储中的某个目录链接到你的项目目录中，这样你就可以直接在项目目录中访问该目录下的文件了（而实际存储在共享存储中）。注意，符号链接的源路径必须是绝对路径，且目标路径必须是相对于当前路径的相对路径。

示例：

```
# 在共享存储中创建目录用于存放实验日志
mkdir /data/zhengjingzhu/exp_results
# 在实验目录中创建符号链接（注意logs不能是一个已存在的目录，否则符号链接会创建到已存在的该目录下）
cd ~/my_project
ln -s /data/zhengjingzhu/exp_results ./logs
```

2. 用户

2.1. 用户登录

集群的所有节点均运行 Ubuntu 操作系统，用户通过 SSH 协议登录集群。用户可以直接在终端中使用 `ssh` 命令登录，或者使用 Xshell，vscode Remote-SSH 等工具登录。登录方式分为密码登录和密钥登录两种方式，推荐使用密钥登录，后者安全性与便捷性均较高，推荐使用后者登录。正常情况下，只能在校园网内才能登录集群。

2.1.1. ssh 命令密码登录

假设用户名为 `zhengjingzhu`，登录节点 IP 为 `10.171.92.20`（节点名与 IP 对应关系参考表 1），则可以在本地终端中使用以下命令登录：

```
ssh zhengjingzhu@10.171.92.20
```

sh

如果是首次登录该节点，系统会提示是否继续连接，输入 `yes` 并回车即可。随后系统会提示输入密码，输入密码时密码不会回显，如果输入错误可以使用退格键删除上一个输入的字符。输入密码后回车即可登录成功。如果输出错误达 3 次，`ssh` 命令将终止，需要重新登录。用户的默认密码为 `Admin@9000`，登录后请务必修改密码，修改密码命令为 `passwd`。

2.1.2. ssh 命令密钥登录

`ssh` 密钥分为公钥和私钥两部分，公钥存放在要登录服务器上，私钥存放在本地电脑上。密钥登录的原理是：当用户尝试登录服务器时，服务器会向用户的电脑发送一个随机生成的字符串，用户的电脑使用私钥对该字符串进行加密，并将加密后的字符串发送回服务器，服务器使用公钥对加密后的字符串进行解密，如果解密后的字符串与最初发送的字符串一致，则登录成功，否则登录失败。

假设用户名为 `zhengjingzhu`，登录节点 IP 为 `10.171.92.20`。

1. 在服务器上创建 `.ssh` 目录和 `authorized_keys` 文件（如果该目录和文件已存在则跳过此步）。注意需要将 `.ssh` 目录权限设置为 700，即仅本用户可以读写和访问，`authorized_keys` 文件权限设置为 600，即仅本用户可读写，否则 SSH 服务器将拒绝（处于安全目的）读取该目录和文件。

```
mkdir -p ~/.ssh
touch ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

sh

2. 在本地电脑上生成密钥对（如果已生成则跳过此步）。执行以下命令后，系统会提示输入文件保存路径，默认路径为 `~/.ssh/id_ed25519`，直接回车即可。随后系统会提示输入密钥密码，可以不设置密码直接回车。`-c` 参数可以添加注释，可以不写该参数。该命令执行完成后，会创建两个文件，`~/.ssh/id_ed25519` 为私钥，`~/.ssh/id_ed25519.pub` 为公钥。请勿将私钥泄露给任何第三方，或者上传到互联网上，否则可能导致你的账号被他人登录，集群管理员亦不会以任何方式向你索取私钥。


```
ssh-keygen -t ed25519 -C "goldenpigeon2000@gmail.com"
```

sh

代码 1 生成 ssh 密钥对

3. 将公钥上传到服务器。打开公钥文件 `~/.ssh/id_ed25519.pub`，复制其中的全部内容。登录服务器后，使用文本编辑器（如 vim, nano 等）打开 `~/.ssh/authorized_keys` 文件，将公钥内容粘贴到该文件的末尾，保存并退出。
4. 使用密钥登录服务器。假设私钥文件路径为 `~/.ssh/id_ed25519`，则可以使用以下命令登录：

```
ssh -i ~/.ssh/id_ed25519 zhengjingzhu@10.171.92.20
```

sh

2.1.3. 配置 ssh 一键登录

如果你不想每次登录都输入完整的用户名，IP 和密钥路径，可以通过配置 `~/.ssh/config`（Windows 上面为 `C:\Users\zhengjingzhu\.ssh\config`）以实现一键登录。

假设用户名为 `zhengjingzhu`，登录节点 IP 为 `10.171.92.20`，私钥路径为 `~/.ssh/id_ed25519`，则可以在 `~/.ssh/config` 中添加以下内容：

```
Host node10
  HostName 10.171.92.20
  User zhengjingzhu
  IdentityFile ~/.ssh/id_ed25519
```

ssh_config

随后即可在终端中使用以下命令一键登录：

```
ssh node10
```

sh

高级用法：

如果不想对每个节点均进行完整的配置，可以使用通配符 `*`，如下所示：

```
Host 10.171.92.*
  IdentityFile ~/.ssh/id_ed25519
  User zhengjingzhu

Host node01
  HostName 10.171.92.1

Host node10
  HostName 10.171.92.20

# ...
```

ssh_config

2.1.4. 使用 VSCode Remote-SSH 登录

在完成密钥配置和 `~/.ssh/config` 配置后，可以使用 VSCode 的 Remote-SSH 插件登录集群，以实现文件编辑和命令执行一体化的远程开发。

1. 安装 VSCode 和 Remote-SSH 插件。
2. 打开 VSCode, 按 F1 (或 Fn+F1), 输入 Remote-SSH: Connect to Host..., 或点击左下角 (如图 1), 选择 Connect to Host...。选择你在 ~/.ssh/config 中配置的节点 (如 node10) 即可登录。

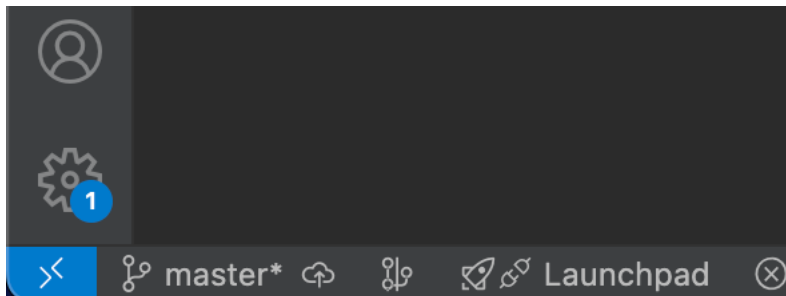


图 1 左下角按钮

3. 如果要临时登录到新的服务器, 可以选择 Remote-SSH: Connect to Host..., 或点击左下角, 选择 Connect to Host...。然后选择 Add New SSH Host..., 输入完整的 ssh 命令 (如 ssh zhengjingzhu@10.171.92.20), 在输入密码后连接上服务器。

2.1.5. 高级：在校园网外登录集群

此操作需要用到 ssh 转发功能, 并且需要在校园网内有一台可以登录集群的电脑 (如工位电脑) 作为跳板机, 和一台可以公网访问的服务器作为中继节点。

记集群节点为 A, IP 为 10.171.92.20, 跳板机为 B, IP 为 10.195.114.51, 中继节点为 C, IP 为 191.9.8.10, 而校园网外的客户端为 D, IP 为 192.168.31.2, 用户名分别为 user_A, user_B, user_C, user_D。

1. 在 B 和 C 上启用 ipv4 转发, 在 B 和 C 的 /etc/sysctl.conf 文件中添加或修改以下内容:

```
net.ipv4.ip_forward=1
```

并执行以下命令使配置生效:

```
sudo sysctl -p
```

sh

2. 在跳板机 B 上执行以下命令 (可以使用 ~/.ssh/config 简化以下命令), 将跳板机 B 的 22 端口转发到中继节点 C 的任意端口上, 例如 2222:

```
ssh -N -R 2222:localhost:22 user_C@191.9.8.10
```

sh

该命令不能中断, 否则转发将失效, 可以使用 tmux 或 screen 等工具保持该命令在后台运行。同时, 可以使用 autossh 保证可以自动重连, 可以参考[此链接](#)。

3. 在中继节点 C 上添加防火墙规则, 允许访问 2222 端口, 并在云服务器的防火墙中允许该端口。
4. 在客户端 D 上执行以下命令访问集群节点 A, 依次输入 C 和 A 的密码即可登录:

```
ssh -J user_C@191.9.8.10:2222 user_A@10.171.92.20
```

sh

5. 为了简化登录命令，可以在 D 的 `~/.ssh/config` 中添加以下内容：

```
Host C
  HostName 191.9.8.10
  User user_C
  Port 2222
  # 使用相应的秘钥文件
  IdentityFile ~/.ssh/id_ed25519

Host A
  HostName 10.171.92.20
  User user_A
  ProxyJump C
  # 使用相应的秘钥文件
  IdentityFile ~/.ssh/id_ed25519
```

随后使用以下命令登录 A 节点：

```
ssh A
```

```
sh
```

代码 2 快捷登录 A 节点

除了上述方法以外，还可以使用自建 VPN 等方式登录集群，具体方法请自行搜索相关资料，可参考[使用 WireGuard 组网实现内网穿透](#)。



提示

本手册作者提供了一个从外网访问集群的途径：

使用 tempuser 用户从 23589 端口登录公网 IP 112.126.76.102，即可登录到一台校园网内的跳板机，再从该跳板机登录集群节点。请注意，该方法存在一定的安全风险，且不保证长期有效，由于网络原因，连接随时可能中断，请谨慎使用。如有问题，请联系本手册作者。

其基础命令为：

```
ssh -P 23589 tempuser@112.126.76.102
```

密码为：temp@9000

2.2. 用户创建（管理员必看）

由于基于 NFS 共享的目录其访问权限是根据用户 uid 和用户组 gid 设置的，因此创建用户时务必要保证用户在各节点上的 uid 和 gid 保持一致，否则会导致共享目录的访问权限发生混乱。

1. 在主节点上创建用户（如已创建，则跳过）。使用 adduser 命令创建用户，而不要使用 useradd，useradd 是“low level”的命令，不推荐使用。而 adduser 命令会自动创建用户的主目录，并设置合适的权限。

```
sudo adduser your_username
```

```
sh
```

随后系统会提示输入用户密码，请使用 `Admin@9000` 作为默认密码。随后需要输入用户全名，房间号，工作电话，家庭电话等信息，可以直接回车跳过这些信息的填写。

2. 查看新用户的 uid 和 gid。使用 `id your_username` 命令查看新用户的 uid 和 gid，并记录下来。例如：

```
id zhengjingzhu
uid=1007(zhengjingzhu) gid=1008(zhengjingzhu) groups=1008(zhengjingzhu)
```

sh

3. 在其他节点上创建同名用户。使用以下命令创建用户，例如：

```
sudo addgroup --gid 1008 zhengjingzhu
sudo adduser --uid 1007 --gid 1008 zhengjingzhu
```

sh

4. 为用户添加合适的用户组，如 `docker` 等。使用以下命令为用户添加用户组，例如：

```
sudo usermod -aG docker zhengjingzhu
```

sh

3. 网络

默认状态下，集群无法访问互联网，这意味着你无法直接在集群上通过互联网安装实验环境，下载实验数据或者在本地浏览器中访问运行在集群节点上的网络服务。你当然可以在无互联网的环境下进行实验，但连接互联网很可能让你的实验事半功倍。

3.1. 集群访问互联网

为了在集群中访问互联网，我们需要利用 ssh 远程端口转发功能，其原理为：将节点上某端口的流量转发到运行在本地的代理服务器中，通过代理服务器访问互联网。为此，需要执行以下步骤：

1. 在本地电脑上安装代理服务器。代理服务器请自行获取，且需要将代理服务器设置为“允许局域网 (LAN) 连接”。我们假设代理服务器的端口为 1145，请根据实际情况调整
2. 在本地电脑上执行以下命令登录以进行远程端口转发（假设用户为 zhengjingzhu，节点 IP 为 10.171.92.20，集群上端口为 1919）

```
ssh -N -R 1919:localhost:1145 zhengjingzhu@10.171.92.20
```

sh

上述命令的作用为：将集群上的 1919 端口的流量转发到本地的 1145 端口（即代理服务器端口）

3. 在集群上设置代理服务器地址

```
export http_proxy=http://localhost:1919
export https_proxy=http://localhost:1919
```

sh

注意地址中均为 http 协议，而不是 https 协议。若需要永久生效，请将上述命令添加入你的 rc 文件中，如 ~/.bashrc，~/.zshrc 等。

对于 git 命令，需要进行特殊设置

```
git config --global http.proxy http://localhost:1919
git config --global https.proxy http://localhost:1919
```

sh

4. 正常使用代理服务器访问互联网

3.2. 访问集群网络服务

为了访问集群中运行的网络服务，如 tensorboard，我们需要利用 ssh 的本地端口转发功能。假设集群节点（10.171.92.20）上运行的网络服务端口为 6006，我们可以通过以下命令将本地端口 8888 转发到集群节点的 6006 端口：

```
ssh -N -L 8888:localhost:6006 zhengjingzhu@10.171.92.29
```

sh

随后，可以在本地的浏览器中访问 <http://localhost:8888> 来访问集群上运行的网络服务。

 高级：在集群外进行上述操作

参考 小节 2.1.5 进行配置后，将前两节命令中的 `zhengjingzhu@10.171.92.29` 换为代码 2 中的 A 即可。

4. 实验

4.1. 实验环境

4.1.1. Python

集群中安装了 [Anaconda](#)，用户可以使用 conda 命令创建和管理 Python 虚拟环境。推荐使用 conda 创建 Python 虚拟环境，并在虚拟环境中安装所需的 Python 包。注意不要在 **base** 环境中安装任何包，以免污染其它用户的环境。

根据实际情况，用户亦可以使用其他包管理器管理虚拟环境，但注意不要直接使用系统级 pip 进行软件包的安装。



提示

如果 http_proxy 和 https_proxy 对 conda 不起作用，可尝试使用以下命令配置代理

```
conda config --set proxy_servers.http http://127.0.0.1:7890
conda config --set proxy_servers.https http://127.0.0.1:7890
```

sh

4.1.2. CUDA

集群中预置了 CUDA 11.6 和 CUDA 12.4，其路径分别为 `/usr/local/cuda-11.6` 和 `/usr/local/cuda-12.4`。用户可以在自己的 profile 文件（如 `~/.bashrc`，`~/.zshrc` 等）中将上述 CUDA 添加到环境变量中。

示例：

```
# ~/.bashrc
export CUDA_HOME=/usr/local/cuda-11.6
export PATH=$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
```

sh

需要切换版本时，只需要修改 `CUDA_HOME` 变量即可。可以使用 `nvcc -V` 命令查看当前 CUDA 版本。

如果系统 CUDA 版本与运行环境所依赖的版本不兼容，用户可以自行以非 root 的形式安装合适的 CUDA 版本。以 CUDA 11.8 为例：

1. 访问 [CUDA 下载页面](#)
2. 点击 Archive of Previous CUDA Releases
3. 点击 CUDA Toolkit 11.8.0
4. 选择 Linux → x86_64 → Ubuntu → 20.04 → runfile(local)（注意不要选成 deb，deb 包只能使用 root 权限安装）
5. 复制下载命令将 CUDA 安装包下载到本地，例如

```
wget https://developer.download.nvidia.com/compute/cuda/11.8.0/local_
installers/cuda_11.8.0_520.61.05_linux.run
```

sh

6. 将 CUDA 安装包上传到集群节点上，例如使用 scp 命令

```
scp cuda_11.8.0_520.61.05_linux.run zhengjingzhu@10.171.92.20:~/
```

sh

7. 登录集群节点，给予安装包可执行权限，并运行安装包

```
chmod +x cuda_11.8.0_520.61.05_linux.run
./cuda_11.8.0_520.61.05_linux.run
```

sh

8. Continue，因为我们不需要安装驱动

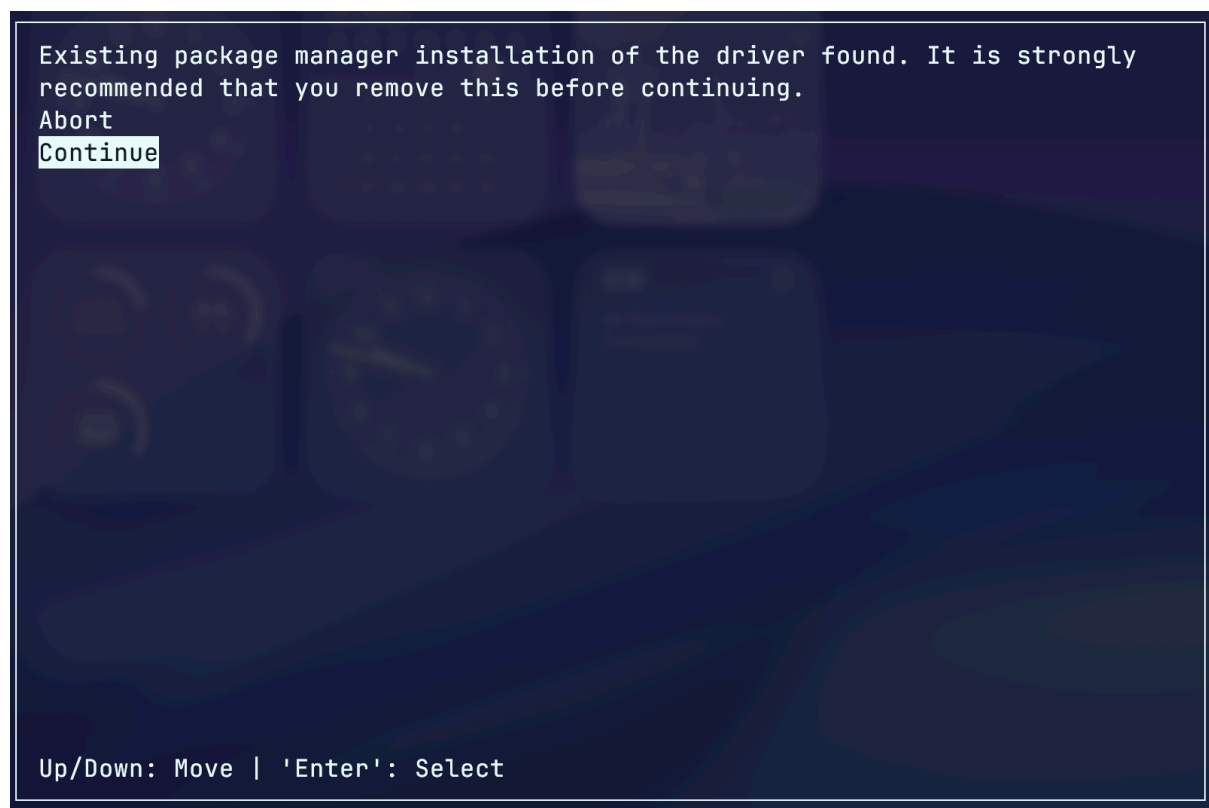


图2 继续

8. accept

9. 进入图 3 页面，取消选择驱动安装；选择 Options → Toolkit Options → Change Toolkit Install Path，进入图 6 页面，选择一个非 root 用户有权限写入的路径，如 /HDD1/zhengjingzhu/cuda-11.8，返回上一级，到图 5 页面，取消 Create symbolic link from /usr/local/cuda，Create desktop menu shortcuts 和 Install manpage documents to /usr/share/man；返回上一级，进入图 4 页面，选择 Library install path (Blank for system default)，进入图 7 页面，写入 /HDD1/zhengjingzhu/cuda-11.8/lib64；返回上两级，选择 Install 开始安装。


```
CUDA Installer
- [ ] Driver
  [ ] 520.61.05
+ [X] CUDA Toolkit 11.8
  [X] CUDA Demo Suite 11.8
  [X] CUDA Documentation 11.8
- [ ] Kernel Objects
  [ ] nvidia-fs
Options
Install

Up/Down: Move | Left/Right: Expand | 'Enter': Select | 'A': Advanced options
```

图 3 取消驱动安装

```
Options
Driver Options
Toolkit Options
Library install path (Blank for system default)
Done

Up/Down: Move | Left/Right: Expand | 'Enter': Select | 'A': Advanced options
```

图 4 选项页面

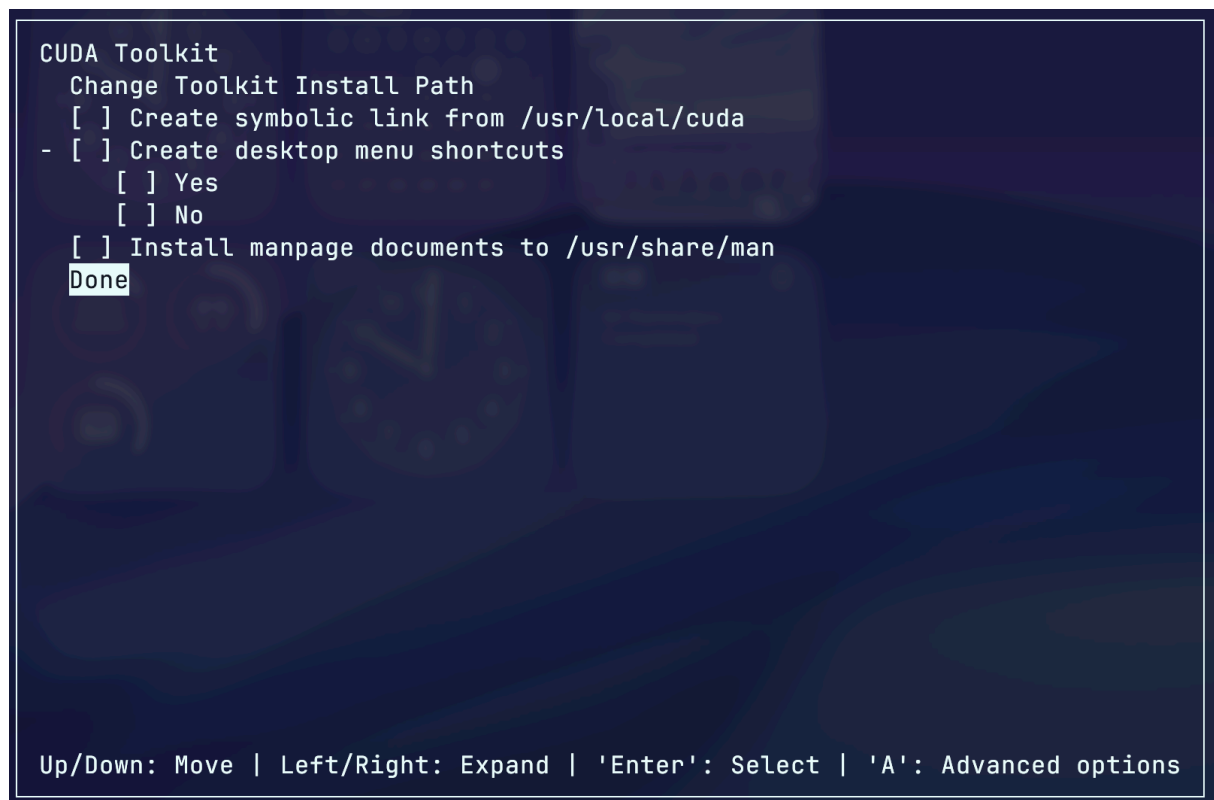


图 5 取消选择

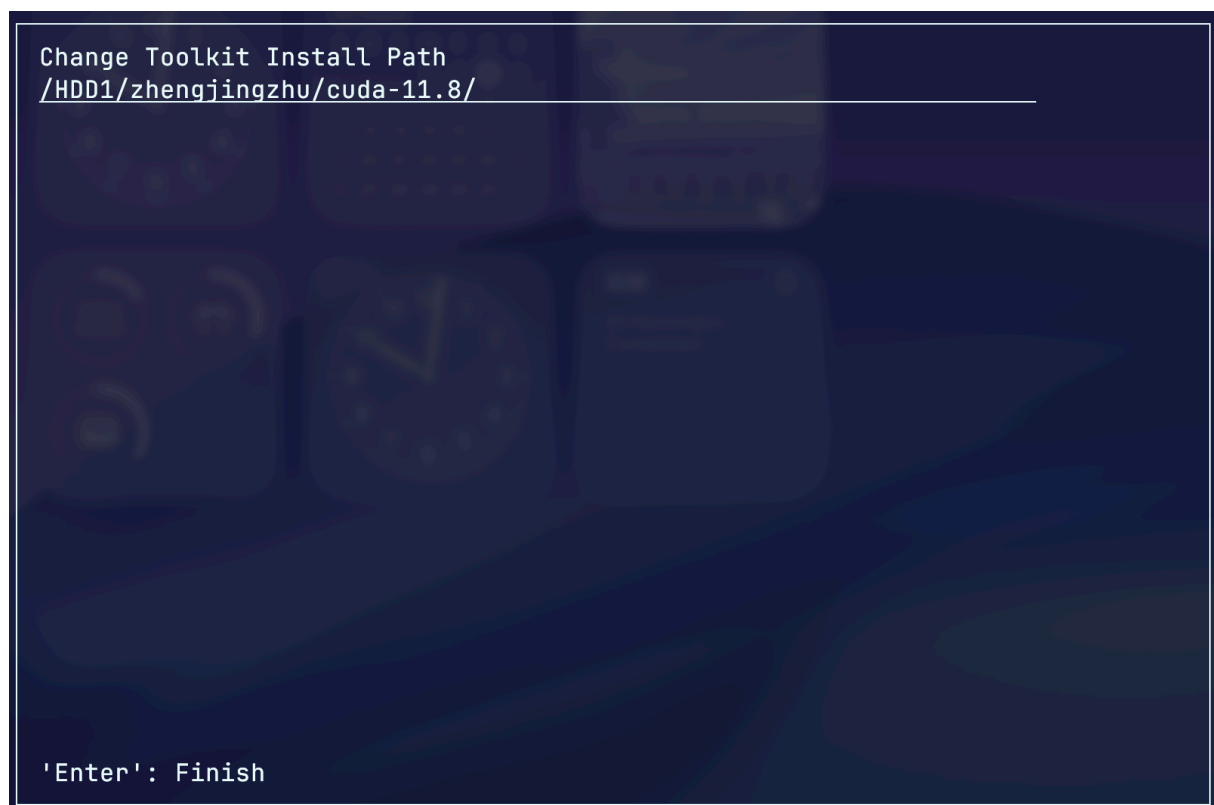


图 6 设置 cuda 安装路径

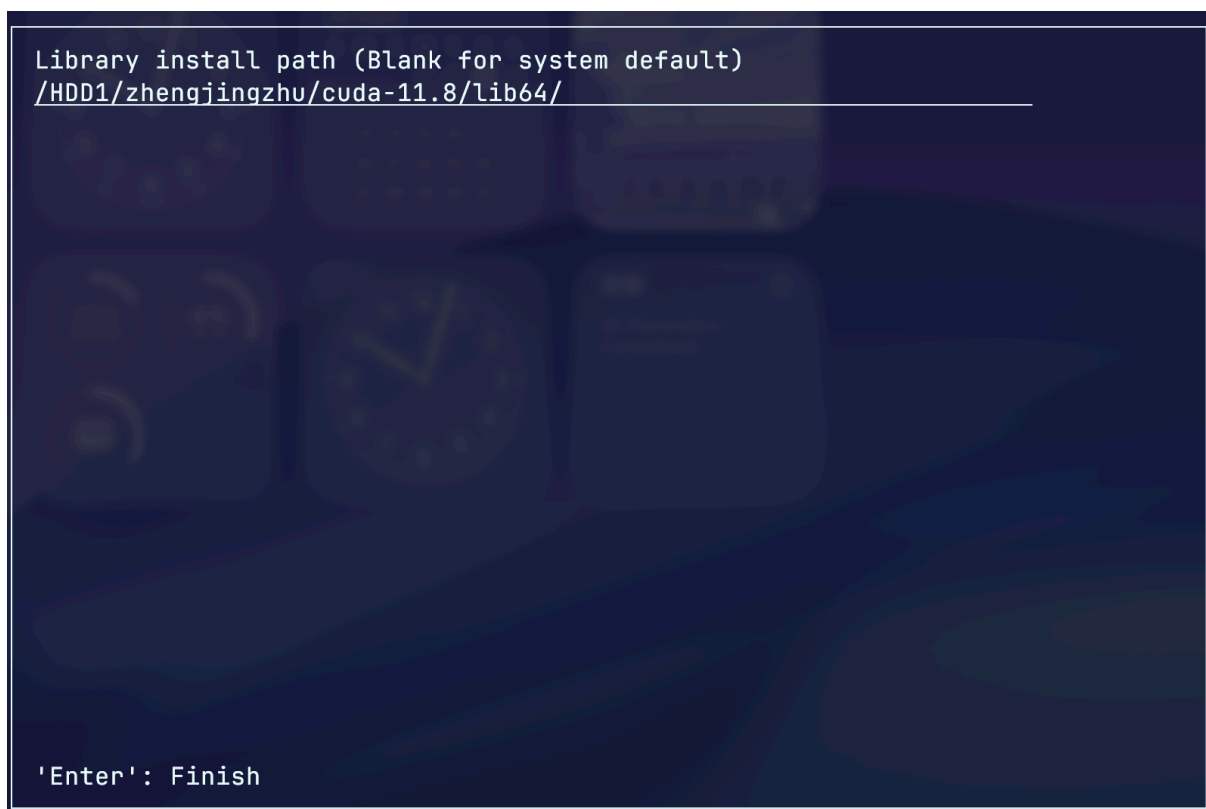


图 7 设置库安装路径

10. 配置环境变量

```
# ~/.bashrc
export CUDA_HOME=/HDD1/zhengjingzhu/cuda-11.8
export PATH=$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH
```

11. 下载 cuDNN。对于 CUDA11.8，一般使用 cuDNN 9.10.2，这是最后支持 CUDA 11 的 cuDNN。访问 [cudnn 下载页面](#)，选择 Archive of Previous Releases → cuDNN 9.10.2，然后选择 Linux → x86_64 → Tarball → 11，使用下载命令将 cuDNN 的 tarball 下载到本地，例如：

```
wget https://developer.download.nvidia.com/compute/cudnn/redirect/cudnn/linux-x86_64/cudnn-linux-x86_64-9.10.2.21_cuda11-archive.tar.xz
```

12. 将 cuDNN 上传到集群节点上

```
scp cudnn-linux-x86_64-9.10.2.21_cuda11-archive.tar.xz zhengjingzhu@10.171.92.20:~/
```

13. 登录集群，解压 cuDNN

```
xz -d cudnn-linux-x86_64-9.10.2.21_cuda11-archive.tar.xz
tar zvf cudnn-linux-x86_64-9.10.2.21_cuda11-archive.tar
```

14. 将头文件和动态链接库复制到 CUDA 的安装目录中的相应位置

```
$ cd cudnn-linux-x86_64-9.10.2.21_cuda11-archive
$ cp include/* /HDD1/zhengjingzhu/cuda-11.8/include/
$ cp lib/* /HDD1/zhengjingzhu/cuda-11.8/lib64/
```

sh

这样，CUDA 和 cuDNN 的安装就完成了

4.1.3. C/C++

鉴于人工智能学院对编程课不够重视😓，许多同学实际上不太熟悉 C/C++ 项目的构建方法。这里将先介绍 C/C++ 相关的一些概念，再讲解如何构建和使用 C/C++ 项目。

4.1.3.1. 概念辨析

源文件 源文件是程序员编写代码逻辑的文件，通常以 .c (C 语言) 或 .cpp (C++) 作为文件后缀。它包含了函数的具体实现、变量的定义等程序主体。程序的入口，即 main 函数，也通常写在源文件中。

头文件 头文件（通常是 .h 或 .hpp 后缀）用于声明函数、类、宏和全局变量等。源文件通过 #include 指令来引入头文件，以便在编译时知晓这些元素的接口和类型，从而正确地使用在其他文件中定义的代码，是模块化编程的关键。

目标文件 目标文件（通常是 .o 或 .obj 后缀）是编译器将单个源文件（如 .cpp）编译后生成的中间产物。它包含了该源文件转换成的机器码、变量和函数符号信息。此时它还不是一个完整的程序，无法独立运行，因为它可能引用了其他源文件中定义的函数，这些引用关系需要链接器来解决。

可执行文件 可执行文件是链接器将一个或多个目标文件以及所需的库文件“链接”起来后，生成的最终文件。它包含了程序运行所需的所有信息，可以被操作系统直接加载到内存中执行。在 Linux 系统中，这类文件普遍遵循 ELF (Executable and Linkable Format) 格式，它是一种标准，详细规定了文件的结构。

静态链接库 静态链接库（在 Linux 中是 .a 后缀，archive）是将多个目标文件打包而成的文件集合。在链接程序时，链接器会从静态库中找出程序所需要的代码和数据，并将它们完整地复制到最终的可执行文件中。程序发布后就不再需要这个库文件了，但缺点是会增大可执行文件的体积。

动态链接库 动态链接库（在 Linux 中是 .so 后缀，Shared Object）是一个独立的文件，它不会在链接时被复制到可执行文件中。链接器只会在可执行文件中记录一个对该库的引用。当程序运行时，操作系统才会加载这个库文件到内存中。多个程序可以共享同一个动态库，从而节省了磁盘和内存空间。

C 运行库 C 运行库 (CRT) 是 C 语言程序运行时所依赖的基础函数集合。由于 C 语言本身非常精简，它不包含输入/输出、内存管理、字符串处理等基本操作。这些功能都由运行库提供，例如我们常用的 printf()、malloc()、strcpy() 等函数。它也是应用程序与操作系统内核沟通的桥梁，让同样的代码可以在不同系统上运行。以 glibc (GNU C Library) 为例，它是 Linux 系统下最核心、最基础的 C 运行库。我们编写的几乎所有 C/C++ 程序在 Linux 上运行时，都会链接并调用 glibc 提供的函数。它实现了 C 语言标准 (ISO C) 和 POSIX 标准中定义的接口，为应用程序提供与内核交互、内存申请、文件读写等一切基础服务。

编译 编译是将人类可读的源文件（.c/.cpp）转换成计算机能够理解的机器码的过程。编译器（如 GCC、Clang）会检查代码的语法，并将每个源文件独立地翻译成一个包含机器指令的中间文件，称为目标文件（.o 或 .obj）。

链接 链接是将编译器生成的多个目标文件（.o/.obj）以及程序所需的库文件（如系统库）合并在一起，创建为一个最终可执行文件的过程。它主要解决不同文件间的函数调用、变量引用等关系，确保它们能正确协同工作。

构建 构建是把源代码转换成最终可执行程序完整流程，是一个宏观概念。它至少包括编译和链接两个主要步骤，有时还包含预处理、资源打包等其他操作。通常我们会使用 Make、CMake 或 IDE 等构建系统来自动化管理整个构建过程。

编译器 编译器是一种软件，其核心任务是将用高级编程语言（如 C/C++）写成的源代码，一次性地翻译成计算机能直接执行的低级机器码。这个过程中它会进行严格的语法检查和代码优化。常见的 C++ 编译器有 GCC、Clang 和微软的 MSVC。

编辑器 编辑器是用于编写和修改纯文本文件的工具。对于编程而言，代码编辑器是专门为编写源代码而设计的，通常提供语法高亮、代码补全、自动缩进等功能来提升编码效率。它只负责“写代码”，不负责编译和运行。例如 Visual Studio Code、Sublime Text、Vim。

IDE IDE (集成开发环境) 是一个功能强大的软件包，它将程序员开发所需的多种工具整合在同一个图形用户界面中。一个 IDE 通常至少包含代码编辑器、编译器和调试器。它提供了一站式的开发体验，使得可以在同一个环境中完成编码、编译、调试和运行程序，极大提高了开发效率。例如 Visual Studio、CLion、Xcode。

4.1.3.2. 编译

本节将以 gcc/g++ 为例（即本集群使用的编译器，分别用于编译 C 语言和 C++ 语言）介绍如何编译和链接一个简单的 C/C++ 程序。

首先创建文件 `hello_world.cpp`

```
#include <iostream>
int main() {
    std::cout << "hello world" << std::endl;
    return 0;
}
```

cpp

代码 3 `hello_world.cpp`

将 `hello_world.cpp` 编译为目标文件：

```
g++ -c hello_world.cpp -o hello_world.o
```

sh

将 `hello_world.o` 链接为可执行文件：

```
g++ hello_world.o -o hello_world
```

sh

执行可执行文件 `hello_world`：

```
./hello_world
```

```
hello world
```

g++实际上是一个编译器工具集合，它集成了预处理，编译，链接等多个工具，因此可以直接一步编译出可执行文件：

```
g++ hello_world.cpp -o hello_world
```

```
sh
```

常见可选参数：

- -g 生成调试信息，允许使用 gdb 对程序进行逐行调试
- -std=c++17 指定 C++ 标准，这里指定为 C++ 17
- -O3 启用最高级优化，可以提高可执行程序的性能
- -l 指定需要链接的库
- -L 指定库文件搜索路径
- -I 指定头文件搜索路径

4.1.3.3. 构建

实际项目中，由于包含大量源文件和头文件并且可能涉及到多种编译选项或第三方库，往往不会直接使用 gcc/g++ 编译，而是使用 make、autotools、cmake 等工具进行构建，通过这种方式简化编译过程，同时还能集成安装等功能。

4.1.3.3.1. make

make 是最基础的构建工具之一，其通过读取名为 Makefile 的文件来管理项目。Makefile 内含一系列规则，每条规则都定义了一个“目标”（如可执行文件、目标文件或需要生成的中间文件等）、生成它所需的“依赖”（如源文件、头文件等），以及生成的“命令”（如编译指令）。当运行 make 时，它会比较文件的时间戳，只重新编译那些依赖项被修改过的目标。这种增量构建的特性避免了对整个项目的重复编译，极大地提升了大型软件的开发效率，是 C/C++ 项目管理中不可或缺的工具。

示例：

项目结构如下：

```
.
├── a.cpp
├── b.cpp
├── b.hpp
└── Makefile
```

各文件内容：

```
#include "b.hpp"

int main(){
    f();
    return 0;
}
```

```
cpp
```

代码 4 a.cpp

```
#include <iostream>
#include "b.hpp"
void f(){
    std::cout << "void f() from b.cpp" << std::endl;
}
```

cpp

代码 5 b.cpp

```
void f();
```

cpp

代码 6 b.hpp

```
all: a

a.o: a.cpp b.hpp
    g++ -c a.cpp -o a.o
b.o: b.cpp b.hpp
    g++ -c b.cpp -o b.o
a: a.o b.o
    g++ a.o b.o -o a

clean:
    rm -rf *.o a

.PHONY: clean
```

make

代码 7 Makefile

构建：执行

```
make
```

sh

输出了编译过程

```
g++ -c a.cpp -o a.o
g++ -c b.cpp -o b.o
g++ a.o b.o -o a
```

最终生成了两个目标文件 `a.o` 和 `b.o` 和可执行文件 `a`。执行 `a`：

```
./a
void f() from b.cpp
```

```
#include <iostream>
#include "b.hpp"
void f(){
    std::cout << "hello" << std::endl;
}
```

cpp

代码 8 b.cpp 修改后

执行 make，输出内容为

```
g++ -c b.cpp -o b.o
g++ a.o b.o -o a
```

可以只重新编译了 b.cpp，并重新进行链接生成 a，而没有重复编译 a.cpp。这在源文件较多的工程中尤为重要，可以大量节省时间。

虽然 Makefile 可以编写各编译目标之间的依赖关系，提高编译效率，但编写 Makefile 较为复杂，特别是在工程较大时。因此，一些大的项目会使用 autotools 和 cmake 等更高级的构建工具，使用这些工具时，只需编写一个比 Makefile 更加简单（虽然在工程变大之后也会非常复杂）的工程文件来定义编译目标、编译选项、外部库等，工具就能自动生成用于实际编译的 Makefile 文件。（即，这些工具并不是直接进行编译，而是用于生成 Makefile 文件（或其他底层文件，如 visual studio 项目文件）的，Makefile 文件才是根基，这就好比高级语言和底层机器语言之间的区别）。

4.1.3.3.2. autotools

Autotools（又称 GNU 构建系统）并非单个工具，而是一个旨在解决软件跨平台可移植性问题的强大工具集，主要由 autoconf、automake 等工具构成。它比 make 更高层，其核心目标是自动生成适应不同 Unix-like 系统的 Makefile 文件。

开发者只需编写一份高阶的描述文件（如 configure.ac 和 Makefile.am），Autotools 就能据此生成一个名为 configure 的智能配置脚本。最终用户在编译前运行此脚本（./configure），它会自动检测用户的系统环境，例如检查需要哪些依赖库、编译器的名称和路径、以及操作系统特性等，然后生成一个为该系统量身定制的 Makefile。

这使得用户可以通过经典的 ./configure、make、make install “三步曲”来轻松地编译和安装软件。Autotools 的核心价值在于，它将开发者从为各种平台编写复杂且难以移植的 Makefile 的繁重工作中解放了出来，是许多经典开源项目构建系统的基石。

下面以介绍 htop 的构建为例，介绍 autotools 工具的使用。之所以选择 htop 为例，是因为 htop 还依赖于另一个需要使用 autotools 构建的库 ncurses。

1. 下载和解压 htop 和 ncurses 源码

```
wget https://github.com/htop-dev/htop/releases/download/3.2.0/htop-3.2.0.tar.xz
wget https://invisible-island.net/archives/ncurses/ncurses-6.5.tar.gz
xz -d htop-3.2.0.tar.xz
tar xvf htop-3.2.0.tar
tar xzvf ncurses-6.5.tar.gz
```

2. 编译 ncurses

首先进行配置

```
cd ncurses-6.5
./configure --prefix=/home/zhengjingzhu/software
```


其中，`--prefix=/home/zhengjingzhu/software` 的作用是设置库的安装路径为 `/home/zhengjingzhu/software` 这是因为默认情况下库会安装到系统的 `/lib` 和 `/include` 等目录中，而我们的用户没有 `root` 权限，不能把文件安装到这些目录，因此需要手动指定一个能访问的安装目录

经过一段时间执行后可以看到如下输出

```
# 上面省略
** Configuration summary for NCURSES 6.5 20240427:

    extended funcs: yes
    xterm terminfo: xterm-new

    bin directory: /home/zhengjingzhu/software/bin
    lib directory: /home/zhengjingzhu/software/lib
    include directory: /home/zhengjingzhu/software/include/ncursesw
    man directory: /home/zhengjingzhu/software/share/man
    terminfo directory: /home/zhengjingzhu/software/share/terminfo

** Include-directory is not in a standard location
```

说明配置已经成功，同时可以注意到项目中已经生成了 `Makefile` 文件，现在就可以正式构建了

```
make -j32
```

sh

`-j32` 的含义是使用 32 线程并行编译，不给这个参数编译就是串行的，速度会很慢。线程数最好不要超过 `cpu` 核心数量。

最后安装该库

```
make install
```

sh

3. 编译 htop

同样先进行配置

```
cd ..
cd htop-3.3.0
./configure CFLAGS="-I/home/zhengjingzhu/software/include" LDFLAGS="-L/home/zhengjingzhu/software/lib" --prefix=/home/zhengjingzhu/software
```

sh

这里的配置除了指定安装目录以外，还指定了头文件目录和库文件目录，否则 `htop` 的 `configure` 脚本会因为找不到 `ncurses` 库而报错。

最后，构建和安装

```
make -j32
make install
```

sh

这样，htop 就安装完毕了，可以将 `/home/zhengjingzhu/software/` 加到 `PATH` 中，然后使用 `htop` 命令了。

4.1.3.3.3. CMake

CMake 是一个现代、开源且高度跨平台的构建系统生成器，被广泛视为 Autotools 的继任者(Autotools 在大项目中异常复杂)。它本身并不直接编译代码，而是旨在解决跨平台构建的难题。开发者只需在项目中编写一个名为 `CMakeLists.txt` 的文本文件，用其简洁的专用语言来描述项目结构、源文件、依赖关系和构建目标。

当用户构建项目时，首先运行 `cmake` 命令。`cmake` 会解析 `CMakeLists.txt`，自动检测系统环境（如编译器、第三方库等），并为当前平台生成一套原生的构建文件。例如，在 Linux 上它会生成 `Makefile`，在 Windows 上则可以生成 Visual Studio 的工程文件（`.sln`），在 macOS 上则可生成 Xcode 项目。之后，用户便可使用这些平台原生的构建工具（如 `make`、Visual Studio 或 Xcode）来执行实际的编译和链接工作。

下面，以我们做项目时通常会用到的 OpenCV 的编译和使用为例，介绍 CMake 的使用。

1. 下载和解压 opencv 源码

```
wget https://github.com/opencv/opencv/archive/refs/tags/4.12.0.zip
unzip 4.12.0.zip
```

2. 配置

```
cd opencv-4.12.0
cmake -DCMAKE_BUILD_TYPE=Debug -DCMAKE_INSTALL_PREFIX=../opencv-4.12.0-build -
DBUILD_opencv_python3=OFF -DBUILD_opencv_python2=OFF -S . -B build
```

这里，`-DCMAKE_BUILD_TYPE=Release` 表示以 Release 模式编译，Release 与 Debug 的区别在于，Debug 会添加调试信息，且不会开启优化，故编译出来的文件会较大，运行速度会较慢。`-DCMAKE_INSTALL_PREFIX=../opencv-4.12.0-build` 定义了安装目录。`-S .` 定义了 `cmake` 项目的源目录（即主 `CMakeLists.txt`）所在的目录为当前目录。`-B build` 定义了构建的目标目录（即构建、编译产生的文件存放的目录）为当前目录下的 `build` 目录。`-DBUILD_opencv_python3=OFF -DBUILD_opencv_python2=OFF` 表示不要构建对 python 的支持。详细构建选项可以参考[文档](#)。

经过一段时间等待后，输出

```
# 上面省略
-- Configuring done (21.7s)
-- Generating done (0.2s)
-- Build files have been written to: /home/xxx/OpenCV/opencv-4.12.0/build
```

说明配置成功

3. 构建和安装

```
cmake --build build -j32
```

其中，`--build build` 表示需要构建的目录，`-j32` 表示同时启动的编译线程数。

经过漫长的等待后，观察到输出

```
# 上省略
[100%] Linking CXX executable ../../bin/opencv_test_gapi
[100%] Built target opencv_test_gapi
```

说明构建完成。接下来安装

```
cmake --install build
```

sh

安装完毕后，可以看到，opencv 相关文件均已安装到 `../opencv-4.12.0-build` 目录中

```
../opencv-4.12.0-build
├── bin
├── include
├── lib
└── share
```

4. 使用 OpenCV 进行开发

```
#include <iostream>
#include <opencv2/opencv.hpp>

using namespace cv;

int main(int argc, char** argv )
{
    if ( argc != 2 )
    {
        std::cout << "usage: ./main <Image_Path>" << std::endl;
        return -1;
    }

    Mat image;
    image = imread( argv[1], IMREAD_COLOR );

    if ( !image.data )
    {
        std::cout << "No image data" << std::endl;
        return -1;
    }
    std::cout << "width: " << image.cols << std::endl;
    std::cout << "height: " << image.rows << std::endl;
    return 0;
}
```

cpp

代码 9 main.cpp

该程序读取了一张图像，并打印其宽和高。

```
# cmake最低版本
cmake_minimum_required(VERSION 3.10)

# 项目名
project (example)

# C++标准为C++17
set(CMAKE_CXX_STANDARD 17)

# Release模式
set(CMAKE_BUILD_TYPE Release)

# 指定opencv安装目录（如果不是默认目录的话）
set(OpenCV_DIR "../opencv-4.12.0/lib/cmake/opencv4")

# 让cmake加载OpenCV的配置
find_package(OpenCV REQUIRED)

# 添加源文件到可执行文件编译目标
add_executable(main main.cpp)

# 指定需要的头文件
target_include_directories(main PRIVATE ${OpenCV_INCLUDE_DIRS})

# 指定需要的库
target_link_libraries(main PRIVATE ${OpenCV_LIBS})
```

cmake

代码 10 CMakeLists.txt

配置和构建

```
cmake -S . -B build
cmake --build build
```

sh

针对图 8 运行

```
./build/main ./000000_0_gt.png
```

sh

输出

```
width: 1600
height: 1066
```

4.2. 实验后台执行

在调试代码时，可以直接在终端中输入训练的命令，执行训练过程，但此时实验仅在前台执行，当 ssh 连接断开或关闭终端窗口后，实验将被中断。为了将实验挂在服务器后台持续执行，可以使用 SLURM 集群调度工具，或者使用终端复用器等多种方式进行。不推荐使用 nohup，因为没法实时显示标准输出。



图 8 000000_0_gt.png

4.2.1. 基于 SLURM 集群任务调度工具的实验（待补充）

SLURM 是一种强大的集群任务调度工具，此工具允许将实验提交到任务队列中，等待系统调度执行，且可以进行多卡多节点的分布式训练。集群集成了此工具，但是没用过，故用法此处略，待补充。

4.2.2. 基于终端复用器的实验

通常可以使用 screen 和 tmux 等终端复用器将实验挂到后台执行。其中，tmux 是更加现代化的终端复用器，推荐使用。以 tmux 为例，介绍一下如何在后台进行实验。

首先使用如下命令，即可开启一个 tmux 窗口，开启窗口后，在其中执行实验命令，在实验执行过程中，即使连接中断，实验也不会中断。

```
tmux new -s myexp
```

sh

要离开当前窗口而不中断正在运行的任务，先按 `Ctrl+B`，再按 `D`，即可离开。

要列出当前存在的所有窗口，使用

```
tmux ls
```

sh

可以得到类似如下的结果

```
copy: 1 windows (created Wed May 28 20:22:31 2025)
data_preparation: 1 windows (created Wed May 28 13:52:04 2025)
download: 1 windows (created Mon Aug 11 14:14:59 2025)
drivingforward: 1 windows (created Fri Sep 5 10:11:27 2025)
lam: 1 windows (created Wed May 28 20:28:06 2025)
myexp: 1 windows (created Wed Oct 8 17:37:18 2025)
st: 2 windows (created Thu Jun 5 02:52:31 2025)
```

这时，若想重新连接先前的窗口，使用

```
tmux attach -t myexp
```

```
sh
```

注意，与 screen 不同，tmux 可以同时有多个会话连接同一个窗口，可以同步显示和控制。这可能带来一个问题，在较小的终端中先连接窗口，然后到较大的终端中连接窗口，后者可能会拥有较小的显示区域，并在四周显示边框，如图 9 所示。

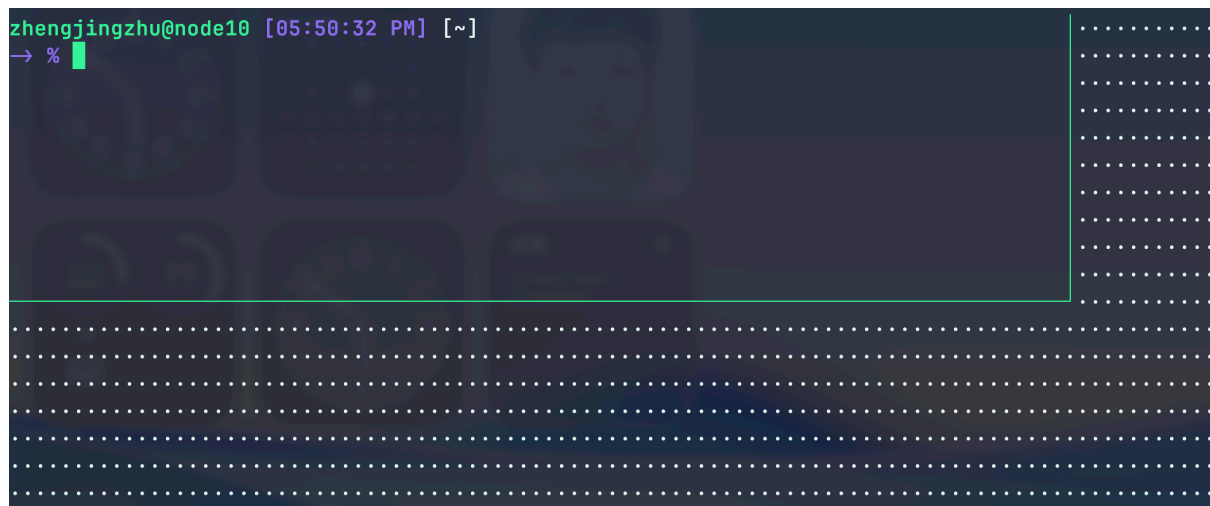


图 9 tmux 显示区域小

此时，只需要将所有的会话都关闭，再在需要使用的终端上重新连接窗口即可。

有时，当实验已经产生了很多输出时，希望翻看前面的输出，可以先按 `Ctrl+B` 再按 `[`，即可进入复制模式(copy mode)，在此模式下，可以通过上下箭头键浏览历史输出。按下 `esc` 键退出该模式。

同一个 tmux 窗口中可以有多多个标签页。要创建新的标签页，先按 `Ctrl+B` 再按 `C` 即可。若要在多个标签页中切换，先按 `Ctrl+B` 再按想切换的标签页的编号所对应的数字键即可。如果相对当前标签页重命名，按 `Ctrl+B` 再按 `,`，可以编辑当前标签页名称，按 `Enter` 确认。

5. Git

” 引述

版本控制（也称为修订控制、源代码控制和源代码管理）是软件工程中的一种实践，用于在计算机文件的历史中控制、组织和跟踪不同版本；主要针对源代码文本文件，但一般适用于任何类型的文件。——维基百科

进行版本控制的优势：

- 可追踪代码修改过程
- 支持多人协作开发，明确代码的修改者，更高效地进行代码合并
- 可进行回溯
- 可将代码备份至远程仓库，防止代码丢失

本手册以 Git 为例讲解对软件项目进行版本控制的方法，Git 最初由 Linux 的作者 Linus Torvalds 开发，现已成为最常用的版本控制软件（大概没有之一）。

🔥 概念辨析

Git 和 GitHub 与 GitLab 是完全不同的概念，前者是一个版本控制工具，而后者是在线的代码托管平台。Git 的作者 Linus 与 GitHub 也没有直接的关联。

此外，GitHub 和 GitLab 虽然同为代码托管平台，但也是完全不同的平台。GitHub 由微软运营，而 GitLab 由 GitLab 公司运营。GitHub 是闭源的平台而 GitLab 是开源的平台且可以在自己的服务器上部署。对于不方便放在 GitHub 上但又需要进行团队协作或备份的代码仓库可以放到自建的 GitLab 上。

GitHub 和 GitLab 并非唯一的代码托管平台，例如 Linux Kernel 就是托管在由 Linux Foundation 自建的平台。

5.1. 概念

- 本地仓库
 - 仓库(repository): 一个由 Git 进行版本控制的项目可以称之为一个仓库。
 - 工作区(Working Directory): 工作区是仓库所包含的能看见的目录，我们在工作区中编写代码。
 - 暂存区(stage): 暂存区包含等待提交的文件，当修改某个文件后，可以将文件添加到暂存区。
 - 提交(commit): 当完成了一个部分或者模块的代码修改后，可以将文件添加到暂存区，并将暂存区中的文件提交到某个分支中，使其成为该分支中可追溯的一部分。Git 会对每个提交计算哈希，可以通过哈希值区分每一个提交。
 - 分支(branch): 分支可以理解为一个开发的时间线，它是有一系列提交组成的一个链表，Git 将其尾指针记为 HEAD（将在后文中讲解）。初始化 Git 仓库时，会自动创建

一条名为“master/main”的分支（在本地创建时，一般为 master，而 GitHub 上默认为 main，这是因为 master 还有“主人”的意思，可能涉及到非裔的政治正确问题😓），称为主分支。随着开发的进行，可以在主分支的某个节点（提交）上分出新的分支，在新分支上进行一些提交后，再将新分支合并到主分支上。这种开发模式是较为推荐的开发模式，一般不建议直接在主分支上进行提交。

- 合并(merge): 将一个分支合并到另一个分支上，就是将前者相比后者新增的提交同步到后者中。在合并过程中，如果 Git 发现两个分支对同一个文件的同一个部分进行了修改，就会发生冲突，用户必须选择保留某个分支的某个部分（可以两个分支都保留一部分，甚至也可以都不要，新写一段）来解决这个冲突，从而完成合并。如果合并过程中解决了冲突，其本身也可以成为一个提交。
- 版本库：版本库存储于 `.git` 目录中，暂存区、分支等都是版本库的一部分。
- 远程仓库
 - 克隆 (clone): 将远程仓库（及其版本库）下载到本地的过程称为克隆。
 - 推送 (push): 将你的提交推送到远程仓库的过程称为推送。
 - 抓取 (fetch): 抓取动作将远程仓库中的新提交下载到本地，但不会自动合并到本地的分支中，需要手动进行合并。
 - 拉取 (pull): 拉取动作将远程仓库中的新提交下载到本地，并自动进行合并。
 - 派生 (fork): 对于一个已存在的远程仓库，可以对其进行 fork，产生一个完全相同的但管理权属于自己的仓库，对 fork 后的仓库进行提交后，提交 PR 请求合并到原有仓库的某个分支中。fork 存在的意义是允许任何人对仓库进行贡献而不用事先将其添加到主仓库的成员列表中。（但实际上也可以成为原仓库的一个备份，应对原仓库删库的情况）
 - Pull Requests: 简称 PR。一般来说，不建议直接 push 主分支，而是将一个开发分支 push 到远程仓库，然后将该分支合并到主分支。若想将自己的开发分支合并到主分支，用户需要在主分支所在仓库提交 PR（没错，PR 可以跨仓库提交，前提是源分支所在仓库是由目的分支所在仓库 fork 产生的），而主分支的维护者会进行审核并进行实际的合并（或者拒绝合并）。如果是自己的仓库，可以自己提交 PR 自己同意合并。
 - Issues: 用户可以在 issues 中提出仓库在使用中遇到的问题，报告 bug 或请求添加新特性，有些仓库会建议在提交 PR 前先提交 issue 进行讨论。

5.2. 项目结构

一个 Git 仓库的结构如下所示：

```
.
├── .git
├── .gitignore
├── LICENSE
└── README.md
```

其中，`.git` 是 Git 仓库的核心，为 Git 的版本库，保存了用于版本控制的所有数据。

`.gitignore` 定义了哪些文件和目录不需要被 Git 追踪，主要会包括数据集、实验日志、可由现有代码生成的文件（如可执行程序，动态链接库，中间数据等）、语言/系统相关的中

间文件和其他不应被追踪的文件。追踪这些文件可能会占用不必要的空间并降低 Git 的解析速度。因此, Git 只应该追踪代码、项目文件和使代码能运行起来的一切必要的文件。Git 不会提示包含在 `.gitignore` 中的文件的修改情况, 也不会将其自动添加到暂存区, 但是可以手动添加这些文件 (虽然并不建议添加)。

`LICENSE` 包含了仓库的许可证, 许可证定义了版权信息, 规定了其他人使用该仓库时的注意事项。

`README.md` 包含仓库的用途和用法。

5.3. 版本控制实战

我们以一个 MNIST 数字分类程序为例展示 Git 实战操作。

- 初始化 Git 仓库

使用以下命令初始化 git 仓库

```
mkdir mnist_classification
cd mnist_classification
git init
```

sh

- 初次提交

添加 `LICENSE`, `README.md` 和 `.gitignore`。

```
touch LICENSE
touch README.md
touch .gitignore
```

sh

代码 11 给出了本仓库的许可证, 该许可证为 MIT 许可证, 属于限制最少的许可证之一, 允许任何人使用、复制、修改、合并、发布、分发、再许可和/或出售软件的副本, 只要在软件和软件的所有副本中包含原始版权声明和许可声明即可。

MIT License

Copyright (c) 2025 Golden-Pigeon

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

代码 11 LICENSE

代码 12 定义了不需要被 Git 追踪的文件和目录。其中，`__pycache__/` 是 Python 的中间文件目录，`build/`、`dist/` 和 `wheels/` 是 Python 打包时生成的目录，`*.egg-info` 是 Python 打包时生成的文件，`.venv` 是 Python 虚拟环境目录，`.vscode` 是 Visual Studio Code 的配置目录。这些目录和文件要么是可以根据现有代码生成的，要么是与代码无关的配置文件，故不需要被 Git 追踪。

```
__pycache__/  
build/  
dist/  
wheels/  
*.egg-info  
  
.venv  
  
.vscode
```

exclude

代码 12 .gitignore

代码 13 给出了本仓库的用途和用法。

```
# git版本控制实战—MNIST数字分类
```

markdown

本项目实现了一个简单的MNIST数字分类程序，使用PyTorch框架进行构建和训练。

代码 13 README.md

现在，可以使用 `git status` 查看当前仓库的状态。

```
On branch master
```

```
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        LICENSE
        README.md

nothing added to commit but untracked files present (use "git add" to track)
```

可以看到，Git 提示我们当前分支为 master，且还没有任何提交。同时，Git 也提示我们三个未被追踪的文件。我们可以将这些文件添加到暂存区，然后进行提交。

```
# 或者 git add . 表示自动添加所有文件
git add .gitignore LICENSE README.md
git commit -m "initial commit."
```

sh

git commit 的 -m 参数指定了该提交的说明，建议对所有的提交给出简洁而明确的说明，以便日后回溯时能清楚地知道每个提交的作用。如果一次修改了多个文件，可以根据修改的内容拆分成多个提交，每个提交只包含一个功能或模块的修改，这样可以使每个提交的说明更加简洁明了。

再次执行 git status，得到以下输出

```
On branch master
nothing to commit, working tree clean
```

这说明当前工作区和版本库是同步的，没有任何需要进一步提交的修改。

- 创建虚拟环境

在本示例中，我们使用 uv 管理虚拟环境。

```
uv init --python=3.13
uv sync
uv add torch torchvision
# 激活环境
source .venv/bin/activate
```

sh

通过 git status 可以看到，uv 创建了 .python-version，hello.py，pyproject.toml 和 uv.lock 四个文件，此外，还有一个虚拟环境目录 .venv，但是由于我们已将该目录添加入 .gitignore 中，所以其并未出现在 git status 的输出中，也不会被自动添加。

```
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .python-version
        hello.py
        pyproject.toml
```

```
uv.lock
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

我们把 `hello.py` 改为 `main.py`，并添加所有文件，进行提交。

```
mv hello.py main.py
git add .
git commit -m "create virtual environment."
```

sh

我们可以使用 `git log` 查看一下提交日志。

```
commit 6e2b913703c63fc5cdcaa6cf205dac9b51e9e2e2
Author: Golden-Pigeon <goldenpigeon2000@gmail.com>
Date:   Wed Oct 15 23:39:10 2025 +0800

    create virtual environment.

commit 39772f9572882f74314ff002361ddb3ad4008a9d
Author: Golden-Pigeon <goldenpigeon2000@gmail.com>
Date:   Wed Oct 15 23:28:28 2025 +0800

    initial commit.
```

gitlog

可以看到，我们此前依次提交了两次，最近的一次提交的哈希值为 `aa6c2496...`，该提交的说明为 `create virtual environment.`，而上一次提交的哈希值为 `e3408789...`，该提交的说明为 `initial commit.`。此外，可以看到每个提交都包含了作者和时间等信息。

- 编写 `mnist` 分类代码

修改 `main.py`

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

batch_size = 64
lr = 1e-3
epochs = 5
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")
```

python

```

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(root="./data", train=True, download=True,
transform=transform)
test_dataset = datasets.MNIST(root="./data", train=False, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.fc1 = nn.Linear(9216, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = Net().to(device)
optimizer = optim.Adam(model.parameters(), lr=lr)
criterion = nn.CrossEntropyLoss()

for epoch in range(1, epochs + 1):
    model.train()
    total_loss = 0
    for data, target in train_loader:
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch}: Train loss = {total_loss / len(train_loader):.4f}")

model.eval()
correct = 0

```

```

test_loss = 0
with torch.no_grad():
    for data, target in test_loader:
        data, target = data.to(device), target.to(device)
        output = model(data)
        test_loss += criterion(output, target).item()
        pred = output.argmax(dim=1)
        correct += pred.eq(target).sum().item()

test_loss /= len(test_loader)
accuracy = 100. * correct / len(test_dataset)
print(f"Test loss: {test_loss:.4f}, Accuracy: {accuracy:.2f}%")

```

执行 `python main.py`，确认可以跑通。运行结束后，执行 `git status`，可以看到，我们修改了 `main.py`，并且下载了数据集到 `data/` 目录中。

```

On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   main.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        data/

no changes added to commit (use "git add" and/or "git commit -a")

```

显然，我们不希望 Git 追踪 `data/` 目录中的数据集文件，因此我们需要将其添加到 `.gitignore` 中。但我们先假设我们忘了添加 `data/` 到 `.gitignore` 中，直接将所有文件添加到暂存区并提交。

```

git add .
git commit -m "add mnist classification code and dataset."

```

此时我们想起来，我们忘了将 `data/` 添加到 `.gitignore` 中，导致 `data/` 目录中的数据集文件也被添加到了版本库中。我们可以执行以下命令

```

git reset --soft HEAD~1

```

前面我们提到过，`HEAD` 表示当前分支的最新提交，`HEAD~1` 表示当前分支的上一个提交。`git reset --soft HEAD~1` 的作用是将当前分支的最新提交回退到上一个提交，但保留工作区和暂存区的修改。执行该命令后，我们可以看到，当前分支已经回退到了上一个提交，但 `main.py` 仍然被修改，且 `data/` 目录仍然存在。（如果我们希望同时撤销修改，可以将 `--soft` 改为 `--hard`）

现在，将 `data/` 添加到 `.gitignore` 中后重新提交。

```
echo "data/" >> .gitignore
git add .
git commit -m "add mnist classification code and dataset."
```

sh

- 建立分支进行特性(feature)开发

当我们想给我们的程序添加一个新特性时，建议在主分支上创建一个新的分支，在新分支上进行开发，完成后再将新分支合并到主分支上。这样可以使主分支保持稳定，且可以在新分支上进行多次提交，便于回溯和修改。这里，我们给代码加一个进度条功能。

首先新建分支

```
git checkout -b dev-progress-bar
```

sh

该命令的作用是首先从当前分支创建一个名为 `dev-progress-bar` 的分支，并切换(checkout)到该分支。如果只想创建而不切换，可以使用 `git branch dev-progress-bar`。如果想指定原分支，可以使用 `git checkout -b dev-progress-bar master`，表示从 master 分支创建新分支。

在虚拟环境中添加 `tqdm` 库。

```
uv add tqdm
```

sh

修改 `main.py`，添加进度条功能。

```
# 在文件开头添加
from tqdm import tqdm
# 在训练循环中添加进度条
pbar = tqdm.trange(1, epochs + 1, position=0)
pbar.set_description(f"Epoch {0:02d} Train loss: {0 / len(train_loader):.4f}")
for epoch in pbar:
    model.train()
    total_loss = 0
    for data, target in tqdm.tqdm(train_loader, position=1, leave=False):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    pbar.set_description(f"Epoch {epoch:02d} Train loss: {total_loss / len(train_loader):.4f}")
# 在验证循环中加入进度条
for data, target in tqdm.tqdm(test_loader):
    ... # 其余代码不变
```

python

运行 `python main.py`，确认可以跑通。运行结束后，执行 `git status`，可以看到，我们修改了 `main.py`，并且由于新增了 `tqdm` 库，`pyproject.toml` 和 `uv.lock` 也发生了变化。接下来，添加文件并提交：

```
git add .
git commit -m "add progress bar feature."
```

sh

- 合并分支

现在，特性分支的开发已经完毕，接下来，我们需要将该分支合并到主分支上。首先，切换回主分支

```
git checkout master
```

sh

提示

切换回主分支后，特征分支中对文件做出的修改会被撤回（即打开文件后会发现修改没了），因为主分支中并未修改文件。也就是说，Git 可以在多个分支中保存同一个文件的多个版本，切换分支时会自动切换到该分支对应的文件版本。

然后，执行合并

```
git merge dev-progress-bar
```

sh

此时，特性分支中的修改已经被合并入主分支。执行 `git log`，可以看到，主分支中已经包含了特性分支的提交记录。由于我们的合并没有产生冲突，故并没有产生新的合并提交。在本地进行开发时，通常不会出现冲突，但在多人协作开发时，冲突是很常见的。解决冲突的方法将在后文中介绍。

```
commit c42061620182670d3f22f07512106670eb9fa15b (HEAD → master, dev-  
progress-bar)
```

gitlog

```
Author: Golden-Pigeon <goldenpigeon2000@gmail.com>
```

```
Date: Sun Oct 19 15:54:54 2025 +0800
```

```
add progress bar feature.
```

```
commit 2e8dcc246c5e5412619014665150e0a013b77501
```

```
Author: Golden-Pigeon <goldenpigeon2000@gmail.com>
```

```
Date: Thu Oct 16 00:06:01 2025 +0800
```

```
add mnist classification code and dataset.
```

提示

实际开发中，并不一定要先创建分支再进行开发。也可以直接在主分支上进行开发，完成后再创建分支（通常发生在忘记创建新分支的情况下）。这种情况下，修改内容会继承到新分支中，之后再在新分支中提交，并将新分支合并回主分支即可。

- 远程仓库操作（以 GitHub 为例）

- 配置 SSH 认证

在将本地仓库推送到远程仓库前，首先需要配置 SSH 认证，以便 GitHub 可以识别你对远程仓库的权限。假设你已经有一个 GitHub 账号，接下来，按照以下步骤进行配置：

1. 参考代码 1 生成 SSH 密钥对。如已经生成，跳过此步骤。
2. 点击头像，选择“Settings”进入设置页面。

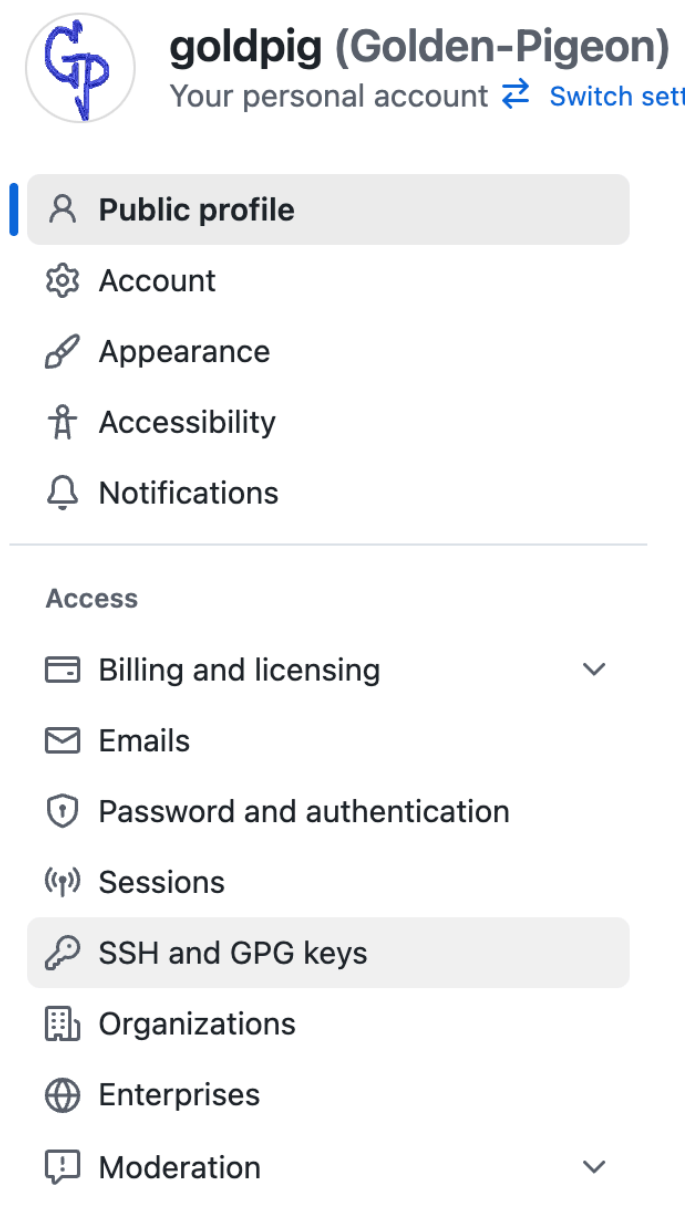


图 10 进入设置页面

3. 点击侧边栏的“SSH and GPG keys”选项，进入 SSH 密钥管理页面。



goldpig (Golden-Pigeon)

Your personal account [Switch sett](#)

 Public profile

 Account

 Appearance

 Accessibility

 Notifications

Access

 Billing and licensing

 Emails

 Password and authentication

 Sessions

 SSH and GPG keys

 Organizations

 Enterprises

 Moderation

图 11 进入 SSH 密钥管理页面

4. 点击“New SSH key”按钮，进入添加 SSH 密钥页面。

Go to your personal profile

New SSH key

图 12 进入添加 SSH 密钥页面

5. 在“Title”中填写该密钥的名称（可任意填写），在“Key”中粘贴公钥内容（如果按照代码 1 创建，则为 `~/.ssh/id_ed25519.pub` 中的内容），然后点击“Add SSH key”按钮完成添加。

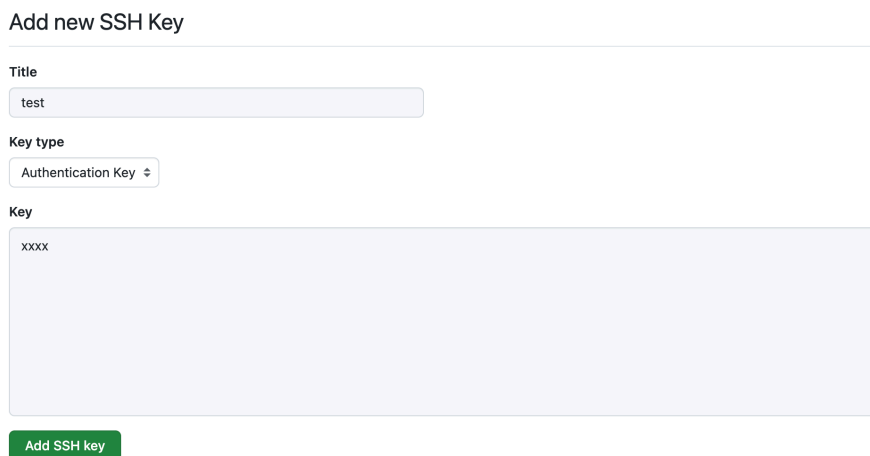


图 13 添加 SSH 密钥

6. 配置 `~/.ssh/config` 文件，添加以下内容

```
Host github.com
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519
```

ssh_config

代码 14 ssh config 配置

7. 测试 SSH 连接

```
ssh -T git@github.com
```

sh

如果看到以下输出，说明 SSH 配置成功

```
Hi Golden-Pigeon! You've successfully authenticated, but GitHub does not
provide shell access.
```

- 将仓库发布到托管平台

首先，在 GitHub 上新建仓库 `mnist_classification`。登录 GitHub 后，点击右上角“Create new... → New repository”按钮。进入新建仓库页面。

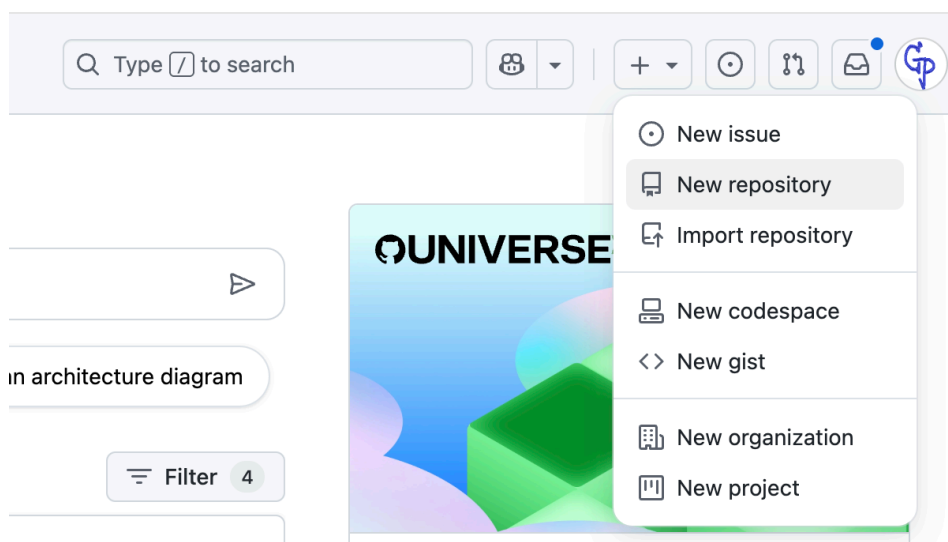


图 14 点击新建仓库

填写仓库名称 `mnist_classification` 和描述, 选择公开仓库(public), 其它的都不要(因为我们已经有现成的仓库了), 然后点击“Create repository”按钮。

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 **General**

Owner * Golden-Pigeon / **Repository name *** `mnist_classification`
✔ `mnist_classification` is available.

Great repository names are short and memorable. How about `fluffy-parakeet`?

Description
`git实战`
5 / 350 characters

2 **Configuration**

Choose visibility * Public
Choose who can see and commit to this repository

Start with a template No template
Templates pre-configure your repository with files.

Add README Off
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore No .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license No license
Licenses explain how others can use your code. [About licenses](#)

Create repository

图 15 新建仓库页面

现在, 我们已经在 GitHub 上创建了一个空的远程仓库, 接下来, 我们可以依照仓库中的提示, 将本地仓库关联到该远程仓库, 并将本地的提交推送到远程仓库中。(注意点击按钮选择 SSH 方式进行关联)

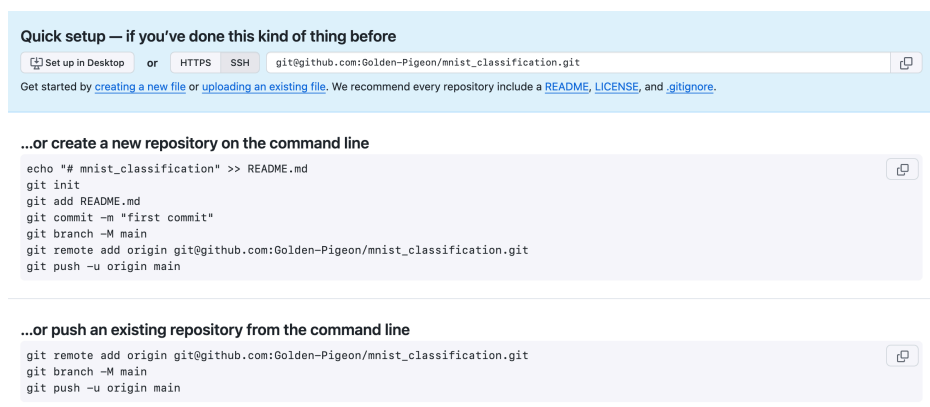


图 16 将本地仓库关联到远程仓库并推送

```
# 设置远程仓库，origin为远程仓库的代号
git remote add origin git@github.com:Golden-Pigeon/mnist_classification.git
# 将当前分支（这意味着必须先把当前分支切为master）改名为main（因为GitHub上默认的主分支名称为main）
git branch -M main
# 将本地的main分支推送到远程仓库的main分支上，并将本地的main分支与远程的main分支关联，此后就可以直接git push
git push -u origin main
```

► 提交 PR

假设我们想给仓库做一些修改，但并不是该仓库的成员，故需要先 fork 该仓库，然后在 fork 后的仓库中进行修改，完成后再提交 PR 请求将修改合并到原仓库中。

这里我们借用了 lmszs 的账号进行演示。

首先，登录 lmszs 账号，进入 mnist_classification 仓库页面，点击右上角的“Fork”按钮，然后直接点击“Create fork”，将该仓库 fork 到自己的账号下。

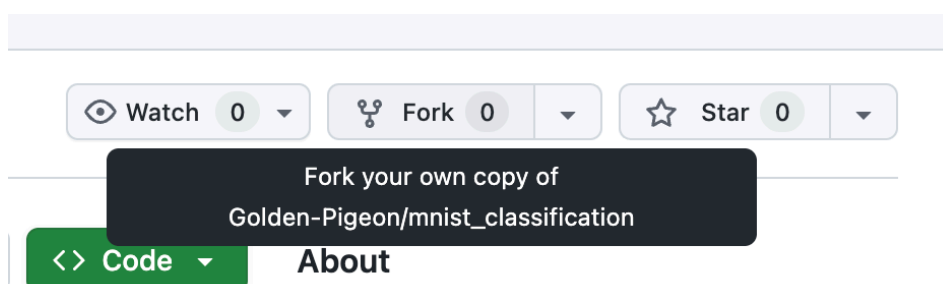


图 17 Fork 仓库

现在，我们已经在 lmszs 账号下拥有了一个 mnist_classification 仓库的副本，接下来，我们可以对其进行一些修改。对于修改内容不多情况，可以直接在 GitHub 网页上进行修改。

我们对 README.md 进行修改，添加程序的运行方式：

点击 README.md 文件，然后点击铅笔图标，进入编辑模式。

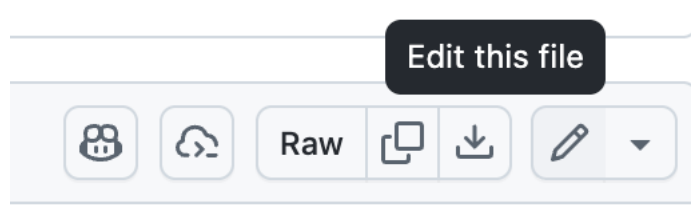


图 18 编辑 README.md

```
# git版本控制实战—MNIST数字分类
本项目实现了一个简单的MNIST数字分类程序，使用PyTorch框架进行构建和训练。

## 如何运行
本项目使用uv管理Python虚拟环境。首先参考[此链接](https://docs.astral.sh/uv/#installation)安装uv。

然后克隆仓库，创建虚拟环境并安装依赖：
```sh
git clone https://github.com/Golden-Pigeon/mnist_classification.git
cd mnist_classification
uv sync
source .venv/bin/activate
```

执行：
```sh
python main.py
```
```

然后点击 `commit changes` 按钮，并输入提交信息，完成提交。

Commit changes

Commit message

Update README with running instructions

Extended description

Added instructions for running the MNIST classification project using uv.

Message and description suggested by Copilot.

☒ Commit directly to the main branch

☐ Create a new branch for this commit and start a pull request

[Learn more about pull requests](#)

Cancel

Commit changes

图 19 提交修改

然后回到 fork 后的仓库页面，点击“Contribute → Open pull request”，并创建 PR。

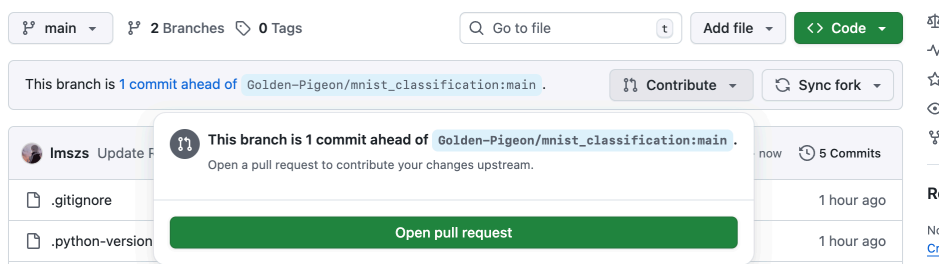


图 20 创建 PR

注意 PR 创建界面的提示，说明该 PR 是从 lmszs 账号下的 main 分支合并到 Golden-Pigeon 账号下的 main 分支的。

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

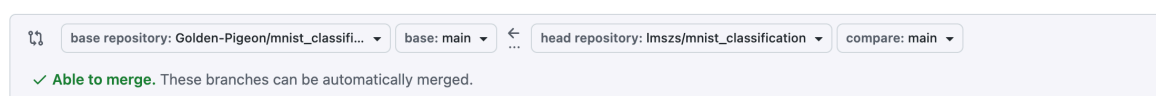


图 21 PR 创建界面

回到作者的账户，可以看到有一个新的 PR 请求，点击进入查看详情。

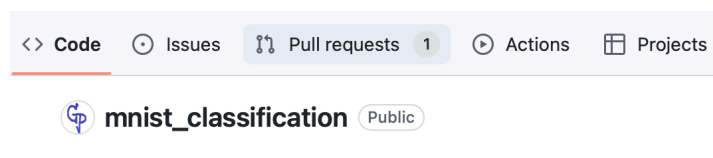


图 22 新的 PR

Update README with running instructions #1

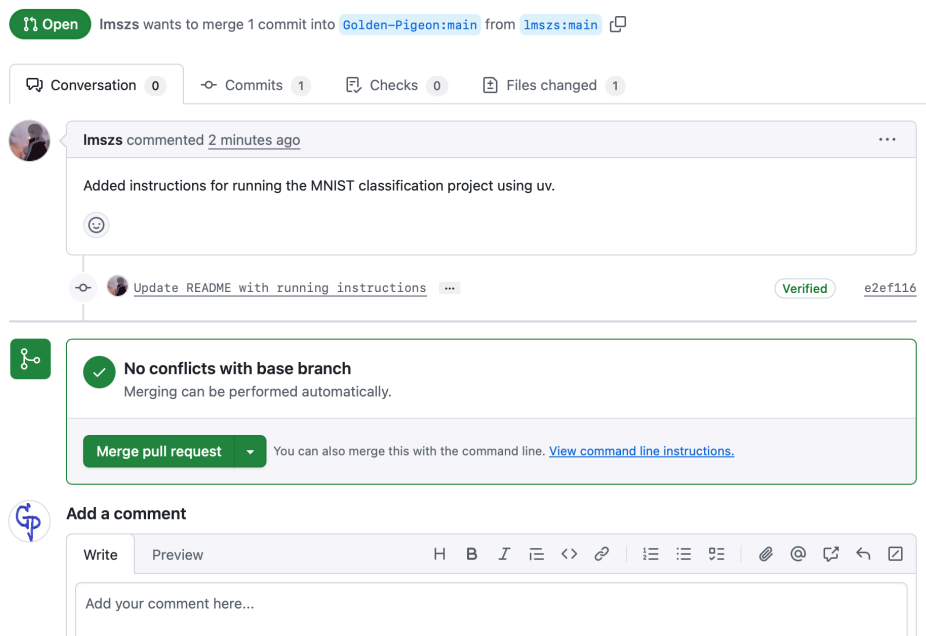


图 23 PR 详情

如果觉得没有问题，可以点击“Merge pull request”按钮，完成合并。合并完成后会自动关闭此 PR。

🔥 提示

此处也可选择 rebase and merge 或 squash and merge 等方式进行合并。前者会将源分支的提交逐个应用到目标分支上，且不会生成合并提交，后者会将源分支的所有提交压缩成一个提交再应用到目标分支上。

回到仓库首页，可以看到 README.md 已经被更新。点击“commit”，可以查看提交历史。

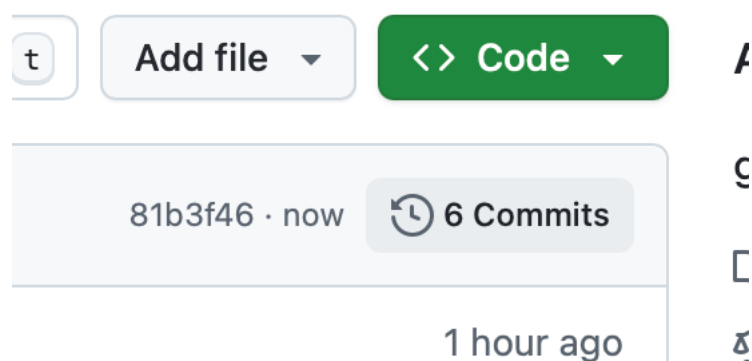


图 24 查看提交历史

可以看到，新增了两条 commit，一条是 lmszs 账号提交的修改，另一条是合并 PR 时自动生成的合并提交。

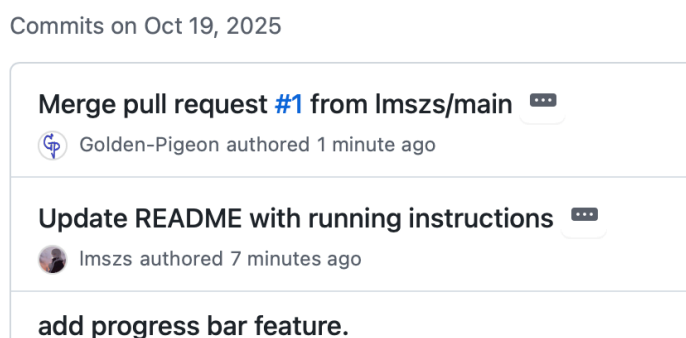


图 25 查看提交详情

🔥 提示

对于更复杂的修改，可以将 fork 后的仓库克隆到本地，在本地进行修改和提交，然后将修改推送到 fork 后的远程仓库中，最后再提交 PR。

▸ 推送新的提交与冲突解决

假设我们在主分支上直接修改了 README.md，并提交了修改。

git版本控制实战—MNIST数字分类

md

本项目实现了一个简单的MNIST数字分类程序，使用PyTorch框架进行构建和训练，并基于uv管理虚拟环境。

git add README.md

sh


```
git commit -m "update README.md"
```

现在，假设我们不知道已经有人在远程仓库中修改了 README.md 并合并了 PR，我们直接将本地的修改推送到远程仓库。

```
git push
```

sh

会推送失败，并输出如下的错误：

```
To github.com:Golden-Pigeon/mnist_classification.git
! [rejected]        main → main (non-fast-forward)
error: failed to push some refs to 'github.com:Golden-Pigeon/
mnist_classification.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

这说明远程仓库中的 main 分支比本地的 main 分支更新，故推送被拒绝。我们需要先将远程仓库中的修改拉取到本地，再进行推送。

```
git pull
```

sh

再次失败，输出

```
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-
hint: rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
```

这是由于我们没有配置应该如何进行拉取。这里我们直接执行以下命令，将默认拉取方式设置为 merge 方式。

```
git config pull.rebase false
```

sh

然后再次尝试拉取，git 再次报错：

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
```

```
Automatic merge failed; fix conflicts and then commit the result.
```

这说明在合并远程仓库的修改时，发生了冲突，且 git 无法自动解决该冲突。我们需要手动解决冲突。

通过 `git status` 可以看到，`README.md` 文件存在冲突，且处于未合并状态。

```
On branch main
Your branch and 'origin/main' have diverged,
and have 1 and 2 different commits each, respectively.
    (use "git pull" if you want to integrate the remote branch with yours)

You have unmerged paths.
    (fix conflicts and run "git commit")
    (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```

通过 `vim` 打开 `README.md` 文件，可以看到冲突的内容被特殊标记了出来。

```
# git版本控制实战—MNIST数字分类
<<<<<<< HEAD
本项目实现了一个简单的MNIST数字分类程序，使用PyTorch框架进行构建和训练，并基于uv管理虚拟环境。

=====
本项目实现了一个简单的MNIST数字分类程序，使用PyTorch框架进行构建和训练。

## 如何运行
本项目使用uv管理Python虚拟环境。首先参考[此链接](https://docs.astral.sh/uv/#installation)安装uv。

然后克隆仓库，创建虚拟环境并安装依赖：
```sh
git clone https://github.com/Golden-Pigeon/mnist_classification.git
cd mnist_classification
uv sync
source .venv/bin/activate
```

执行：
```sh
python main.py
```

>>>>>> 81b3f464eb44a77fb7389038fa3bd052c19792e7
```

其中，<<<<<<< HEAD 到 ===== 之间的内容是当前分支的内容，>>>>>>> 81b3f464eb44a77fb7389038fa3bd052c19792e7 之间的内容是要合并进来的分支的内容。我们需要根据实际情况，修改该文件，去掉冲突标记，并保留我们想要的内容（可以都保留一部分，保留一方或者进行别的修改）。这里，我们保留远程仓库的内容，并删除所有标记行。

git版本控制实战—MNIST数字分类

md

本项目实现了一个简单的MNIST数字分类程序，使用PyTorch框架进行构建和训练。

如何运行

本项目使用uv管理Python虚拟环境。首先参考[此链接](<https://docs.astral.sh/uv/#installation>)安装uv。

v。

然后克隆仓库，创建虚拟环境并安装依赖：

```
```sh
git clone https://github.com/Golden-Pigeon/mnist_classification.git
cd mnist_classification
uv sync
source .venv/bin/activate
```
```

执行：

```
```sh
python main.py
```
```

保存文件后，执行以下命令，标记冲突已解决，并提交修改。

🔥 提示

使用 vim 解决冲突较为麻烦，为了简单起见，可以使用 VSCode 和 PyCharm 等编辑器/IDE 进行更加高效的编辑。

```
git add README.md
git commit -m "resolve merge conflict in README.md"
```

sh

现在，远程仓库的修改已经被合并到本地仓库中，且冲突已经解决。接下来，我们可以将本地的修改推送到远程仓库中。

```
git push
```

sh

推送成功。

提示

如果你已经知道有人更新远程仓库，且希望立即将其拉取到本地仓库，但是你的本地修改并未完成，你可以使用 `git stash` 命令先将本地修改暂存起来，然后拉取远程仓库的修改，再使用 `git stash pop` 命令将之前的修改 pop 回来（如果有冲突一样按照此前的方法解决），最后再进行提交和推送。

5.4. 一个自建的 GitLab 服务器

为了解决实验室成员备份代码、协作开发的需求并满足代码的安全性，作者提供了一个自建的 GitLab 服务器用于代码托管。使用方法如下：

1. 参考小节 B.1. 加入 VPN 网络，这是由于校园网不稳定，校园网 IP 经常变化，VPN 网络 IP 固定。
2. 联系作者创建 GitLab 账号。
3. 浏览器访问 `http://172.16.0.4:9980/users/sign_in` 登录。
4. 在用户设置中配置 SSH Key，并配置 `~/.ssh/config`。参考代码 14。
5. 创建仓库，并与本地仓库关联。注意使用 SSH 方式，注意在提供的 URL 中，不论它写的是什么，都应该把 IP 地址替换为 **172.16.0.4**。
6. 如果是在集群上操作，由于集群无法访问外部互联网（代理服务器只能作用于有域名的 URL，没法直接作用于纯 IP 地址）。故若想使用本 GitLab，应参考小节 3.1 进行转发，只不过这里要转发的是 GitLab 所使用的 9922 端口而不是代理服务器的端口。然后使用 `localhost:9922` 代替 `172.16.0.4:9922`。

Appendix

A. Linux 使用技巧

本章将介绍一些零碎的 Linux 系统使用技巧，以供读者参考。

A.1. 文件操作

1. 列出文件

```
# 列出当前目录下的文件和目录，不包含.开头的文件和目录
ls

# 列出当前目录下所有的文件和目录
ls -a

# 列出文件的详细信息
ls -l

# 列出文件的详细信息，并使用人类可读的形式显示大小
ls -lh

# 列出文件的详细信息，并按时间排序
ls -lt
```

sh

其中，`ls -l` 通常可以简写为 `ll`，故 `ls -lh` 也可写为 `ll -h`，以此类推

2. 创建文件和目录

```
# 创建一个test.txt文件
touch test.txt

# 创建一个目录
mkdir test_dir

# 递归创建目录（用于需要重建多层目录的情况）
mkdir -p dir/subdir/subsubdir
```

sh

3. 软链接

软链接类似于 windows 中的快捷方式

注意以下命令中第一个路径必须为绝对路径

```
ln -s /absolute/path/to/source/file relative/path/to/dst/file
```

sh

4. 复制和移动

```
# 将file1复制为dir下的file2
cp file1 dir/file2

# 将dir1目录复制为dir2目录，若后者存在，则复制为dir2/dir1
cp -r dir1 dir2

# 重命名file1
mv file1 file2

# 将dir1目录移动到dir2目录，若后者存在，则移动到dir2/dir1
mv dir1 dir2
```

sh

```
# 将dir1下的内容同步到dir2下，跳过相同的内容（注意/是必须的）
rsync -av dir1/ dir2/
```

注意，在同一个磁盘分区上移动时，移动可以即时完成，无论目标有多大，而在不同分区之间移动时，需要先把数据复制过去，需要时间。

5. 权限管理

linux 文件的权限分为 r、w、x 权限，分别表示读，写，执行，可使用二进制表示，从左到右三位分别表示 r，w，x，之后转化为八进制。有三种基础的控制粒度，包括当前用户，当前用户组和其他用户，分别记为 u，g，o，故可以使用三个八进制数来表示一个文件的权限。部分场合对文件权限有较严格的限制，比如 ssh 的配置和公钥。目录必须要有可执行权限才能访问。

```
# 给file1赋予当前用户和当前用户组可读写执行权限
chmod 770 file1
# 给file2为当前用户增加可执行权限，其它权限不变
chmod u+x file2
# 给file3为其它用户移除读写权限，其它权限不变
chmod o-rw file3
# 给dir递归设置权限为当前用户可读写
chmod 600 dir -R
# 把file1的所有者改为zhengjingzhu，用户组改为zhengjingzhu
chown zhengjingzhu:zhengjingzhu file1
# 把dir及其子目录和文件的所有者改为zhengjingzhu，用户组改为zhengjingzhu
chown -R zhengjingzhu:zhengjingzhu dir
```

A.2. 压缩与解压

Linux 系统中常见的打包格式包括 tar，gzip，xz，bz2，zip，其中，tar 仅用于将文件和目录打包为一个文件但不压缩，其它格式均为压缩格式。gzip，xz 和 bz2 通常也会和 tar 一起使用，即先将文件和目录打包为一个文件，再压缩该文件。

1. tar

tar 包后缀名为.tar，打包命令如下，可以将多个文件和目录打包进一个 tar 包

```
tar cvf name.tar dir1 file2 ...
```

使用

```
tar xvf name.tar
```

即可解包，可将所有文件提取到当前目录下

若想指定目录，使用

```
tar xvf name.tar -C dir
```

若想只显示包含的文件列表，不实际提取文件，使用

```
tar tvf name.tar
```

```
sh
```

1. tar

tar 包后缀名为.tar，打包命令如下，可以将多个文件和目录打包进一个 tar 包

```
tar cvf name.tar dir1 file2 ...
```

```
sh
```

使用

```
tar xvf name.tar
```

```
sh
```

即可解包，可将所有文件提取到当前目录下

若想指定目录，使用

```
tar xvf name.tar -C dir
```

```
sh
```

若想只显示包含的文件列表，不实际提取文件，使用

```
tar tvf name.tar
```

```
sh
```

2. gzip

gzip 通常和 tar 一起使用，压缩包后缀名为.tar.gz 或者.tgz，打包命令如下，可以将多个文件和目录打包进一个 tar.gz 包

```
tar czvf name.tar.gz dir1 file2 ...
```

```
sh
```

使用

```
tar xzvf name.tar.gz
```

```
sh
```

即可解包，可将所有文件提取到当前目录下

若想指定目录，使用

```
tar xzvf name.tar.gz -C dir
```

```
sh
```

若想只显示包含的文件列表，不实际提取文件，使用

```
tar tzvf name.tar
```

```
sh
```

如果是单纯的 gz，不使用 tar，可以使用 gzip 和 gunzip 命令进行压缩和解压

```
# 生成一个file1.gz，删除原file1
gzip file1
# 生成file1，删除file1.gz
gunzip file1.gz
```

```
sh
```

3. bz2

bz2 通常和 tar 一起使用，压缩包后缀名为.tar.bz2，打包命令如下，可以将多个文件和目录打包进一个 tar.bz2 包

```
tar cjvf name.tar.bz2 dir1 file2 ...
```

sh

使用

```
tar xjvf name.tar.bz2
```

sh

即可解包，可将所有文件提取到当前目录下

若想指定目录，使用

```
tar xjvf name.tar.bz2 -C dir
```

sh

若想只显示包含的文件列表，不实际提取文件，使用

```
tar tjvf name.tar.bz2
```

sh

如果是单纯的 bz2，不使用 tar，可以使用 bzip2 和 bunzip2 命令进行压缩和解压

```
# 生成一个file1.bz2，删除原file1
bzip2 file1
# 生成file1，删除file1.bz2
bunzip2 file1.bz2
```

sh

4. xz

xz 通常和 tar 一起使用，压缩包后缀名为.tar.xz，打包命令如下，可以将多个文件和目录打包进一个 tar.xz 包

```
tar cJvf name.tar.xz dir1 file2 ...
```

sh

使用

```
tar xJvf name.tar.xz
```

sh

即可解包，可将所有文件提取到当前目录下

若想指定目录，使用

```
tar xJvf name.tar.xz -C dir
```

sh

若想只显示包含的文件列表，不实际提取文件，使用

```
tar tJvf name.tar.xz
```

sh

如果是单纯的 xz，不使用 tar，可以使用 xz 命令进行压缩和解压

```
# 将file1压缩为file1.xz，删除原文件
xz file1
# 将file1.xz解压为file1，删除压缩包
```

sh


```
xz -d file1.xz
```

5. zip

zip 包的后缀名为.zip。压缩命令为

```
zip -r name.zip dir1 file2
```

sh

解压命令为

```
unzip name.zip
```

sh

若要指定解压目录，使用

```
unzip name.zip -d dir
```

sh

若想只显示包含的文件列表，不实际提取文件，使用

```
unzip -l name.zip
```

sh

B. 一个自建的基于 Wireguard 的 VPN 网络

B.1. 加入网络

B.1.1. Windows

1. 下载并安装 Wireguard 客户端，地址为 <https://download.wireguard.com/windows-client/wireguard-installer.exe>
2. 打开应用程序后，点击“新建隧道 → 新建空隧道”。
3. 在弹出的编辑器中，初始包含以下内容：

```
[Interface]
PrivateKey = ${客户端私钥}
```

不要动这两行，并添加以下内容（其中 x 需要与作者协商确定）：

```
Address = 172.16.0.x/24

[Peer]
PublicKey = D2W2TvwHd6CnVydmrXQutmnhZ1qIqLbSIQWsTsSApwY=
Endpoint = 112.126.76.102:51820
AllowedIPs = 172.16.0.0/24
PersistentKeepalive = 25
```

此外，记录下该窗口中显示的客户端公钥（注意不是刚刚输入的公钥，刚刚输入的是中继服务器的公钥）。

4. 联系作者，将你的公钥发送给作者，以便作者将你的公钥和 IP 地址加入服务器配置中。

5. 点击连接，即可进入 VPN 网络。

B.1.2. Linux

1. 更改系统配置允许 IP 转发

```
sudo vim /etc/sysctl.conf
```

sh

添加或取消注释以下行：

```
net.ipv4.ip_forward=1
```

然后，执行

```
sudo sysctl -p
```

sh

2. 安装 Wireguard

```
sudo apt install wireguard
```

sh

3. 生成密钥对

```
wg genkey | sudo tee /etc/wireguard/privatekey | wg pubkey | sudo tee /etc/wireguard/publickey
```

sh

4. 查看默认网络接口（我这里为 ppp0）

```
ip -o -4 route show to default | awk '{print $5}'
```

sh

5. 创建配置文件

```
sudo vim /etc/wireguard/wg0.conf
```

sh

内容如下（x 需与作者协商）：

```
[Interface]
PrivateKey = ${/etc/wireguard/privatekey的内容}
Address = 172.16.0.x/24
PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -A FORWARD -o %i -j ACCEPT; iptables -t nat -A POSTROUTING -o ${你的默认网络接口} -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -D FORWARD -o %i -j ACCEPT; iptables -t nat -D POSTROUTING -o ${你的默认网络接口} -j MASQUERADE

[Peer]
PublicKey = D2W2TvwHd6CnVydmrXQutmnhZ1qIqlbSIQWsTsSApwY=
Endpoint = 112.126.76.102:51820
AllowedIPs = 172.16.0.0/24
PersistentKeepalive = 25
```

随后设置配置目录权限

```
sudo chmod 600 /etc/wireguard/*
```

```
sh
```

6. 联系作者，将你的公钥（/etc/wireguard/publickey 文件内容）发送给作者，以便作者将你的公钥和 IP 地址加入服务器配置中。

7. 启动 Wireguard

```
sudo wg-quick up wg0
```

```
sh
```

8. 配置开机自启动

```
sudo systemctl enable wg-quick@wg0
```

```
sh
```

B.1.3. MacOS

参考 Windows 配置。

B.2. VPN 网络注意事项

1. 在此 VPN 网络中，可以访问任何一个节点上的网络服务，如 SSH, FTP, HTTP/HTTPS 服务等。这也意味着你不应该给电脑设置一个过于简单的密码，否则可能被别人通过 SSH 登录。
2. 中继服务器带宽有限，故不要使用此 VPN 进行大规模的数据传输，如上传数据集。

Index of Figures

| | | |
|------|-----------------------|----|
| 图 1 | 左下角按钮 | 10 |
| 图 2 | 继续 | 16 |
| 图 3 | 取消驱动安装 | 17 |
| 图 4 | 选项页面 | 17 |
| 图 5 | 取消选择 | 18 |
| 图 6 | 设置 cuda 安装路径 | 18 |
| 图 7 | 设置库安装路径 | 19 |
| 图 8 | 000000_0_gt.png | 28 |
| 图 9 | tmux 显示区域小 | 30 |
| 图 10 | 进入设置页面 | 41 |
| 图 11 | 进入 SSH 密钥管理页面 | 42 |
| 图 12 | 进入添加 SSH 密钥页面 | 42 |
| 图 13 | 添加 SSH 密钥 | 43 |
| 图 14 | 点击新建仓库 | 44 |
| 图 15 | 新建仓库页面 | 44 |
| 图 16 | 将本地仓库关联到远程仓库并推送 | 45 |
| 图 17 | Fork 仓库 | 45 |
| 图 18 | 编辑 README.md | 46 |
| 图 19 | 提交修改 | 46 |
| 图 20 | 创建 PR | 47 |
| 图 21 | PR 创建界面 | 47 |
| 图 22 | 新的 PR | 47 |
| 图 23 | PR 详情 | 47 |
| 图 24 | 查看提交历史 | 48 |
| 图 25 | 查看提交详情 | 48 |

Index of Tables

| | | |
|-----|--------------|---|
| 表 1 | 集群整体结构 | 5 |
| 表 2 | 集群存储结构 | 5 |

Index of Listings

| | | |
|------|-----------------------|----|
| 代码 1 | 生成 ssh 秘钥对 | 9 |
| 代码 2 | 快捷登录 A 节点 | 11 |
| 代码 3 | hello_world.cpp | 21 |
| 代码 4 | a.cpp | 22 |
| 代码 5 | b.cpp | 23 |
| 代码 6 | b.hpp | 23 |
| 代码 7 | Makefile | 23 |
| 代码 8 | b.cpp 修改后 | 23 |
| 代码 9 | main.cpp | 27 |

| | | |
|-------|----------------------|----|
| 代码 10 | CMakeLists.txt | 28 |
| 代码 11 | LICENSE | 34 |
| 代码 12 | .gitignore | 34 |
| 代码 13 | README.md | 34 |
| 代码 14 | ssh config 配置 | 43 |